

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ
Кафедра математического моделирования и анализа данных

САЗОНЧИК Иван Михайлович

**ОЦЕНКА КАЧЕСТВА ЭНТРОПИИ В БИОЛОГИЧЕСКИХ
ГЕНЕРАТОРАХ СЛУЧАЙНЫХ ЧИСЕЛ НА ОСНОВЕ
ПОВЕДЕНЧЕСКИХ ФАКТОРОВ**

Дипломная работа

Научные руководители:
старший преподаватель
М.А. Казловский

Допущена к защите

«___» _____ 2026 г.

Заведующий кафедрой математического
моделирования и анализа данных
доктор экономических наук, кандидат физико-
математических наук, профессор В.И. Малюгин

Минск, 2026

Белорусский государственный университет
Кафедра математического моделирования и анализа данных

УТВЕРЖДАЮ

Заведующий _____ кафедрой

(подпись) (фамилия, инициалы)

_____ 20__ г.

ЗАДАНИЕ НА ДИПЛОМНУЮ РАБОТУ

Обучающемуся Сазончику Ивану Михайловичу

(фамилия, собственное имя, отчество (если таковое имеется))

Курс _____ 4 _____ Учебная группа _____ 9 _____

Специальность _____ компьютерная безопасность _____

Тема дипломной работы Оценка качества энтропии в биологических генераторах случайных чисел на основе поведенческих факторов

(наименование темы)

Утверждена приказом ректора БГУ от _____ № _____

Исходные данные к дипломной работе

1. Rukhin A. et al. A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications.

NIST Special Publication 800-22. –2010. – 131 с.

2. Агиевич, С. В. Криптографические методы / С. В. Агиевич//НИИ прикладных проблем математики и информатики БГУ.

3. Ямпольский, В. В. Биометрические методы и средства идентификации личности/ В. В. Ямпольский, Д. В. Михеев. — М.

Перечень подлежащих разработке вопросов или краткое содержание расчётно-пояснительной записки:

1. Подготовить обзор научной литературы, определить цели и задачи исследования.
2. Разработать программный комплекс для захвата и обработки окулоmotorных и голосовых данных.
3. Реализовать алгоритмы извлечения, накопления и смешивания энтропии из двух каналов.
4. Реализовать оценку статистического качества генерируемых последовательностей (NIST SP 800-90B).
5. Провести компьютерный эксперимент и анализ результатов.

Перечень графического материала (с точным указанием обязательных чертежей и графиков)

Консультанты по дипломной работе (с указанием разделов, по которым они консультируют)_____

Примерный календарный график выполнения дипломной работы

Дата выдачи задания_____

Сроки сдачи законченной дипломной работы_____

Руководитель дипломной работы _____
(подпись) (инициалы, фамилия)

Подпись обучающегося _____

Дата_____ 20__ г.

Проинформирован о недопустимости привлечения третьих лиц к выполнению дипломной работы, плагиата, фальсификации или подлога материалов.

(подпись)

(инициалы, фамилия)

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	10
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ БИОЛОГИЧЕСКИХ ГЕНЕРАТОРОВ СЛУЧАЙНЫХ ЧИСЕЛ	13
1.1 Классификация источников энтропии	13
1.2 Особенности источников, основанных на зрительной активности оператора	14
1.3 Голос как источник энтропии: биофизические основания	16
2 РАЗРАБОТКА ИНТЕРФЕЙСА ПРОГРАММНОГО КОМПЛЕКСА	20
2.1 Блок управления трекингом (Control Card)	20
2.2 Блок стимулов (Triggers Card)	20
2.3 Блок генерации ключа (Key Card)	21
2.4 Панели статистики (Statistics & Audio Widgets)	21
3 РЕАЛИЗАЦИЯ АЛГОРИТМОВ КОМПЛЕКСА ДЛЯ СБОРА ЭНТРОПИИ	22
3.1 Архитектура системы	22
3.2 Алгоритмы трекинга визуального сигнала	23
3.3 Алгоритмы сбора энтропии из визуального сигнала . . .	27
3.4 Алгоритмы сбора энтропии из голосового сигнала . . .	29
3.5 Смешивание голосовой и глазной энтропии	33
3.6 Реализация	34
4 ОЦЕНКА КАЧЕСТВА ЭНТРОПИИ ПРИЛОЖЕНИЯ	35
4.1 Тестовый стенд	35
4.2 Инструменты оценки	36
4.3 Критерии успешности	37
4.4 Результаты тестирования	38
ЗАКЛЮЧЕНИЕ	41
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	43

ПРИЛОЖЕНИЕ А 44

ПРИЛОЖЕНИЕ Б 45

ПРИЛОЖЕНИЕ В 57

Реферат

Дипломная работа: 43 страницы, 4 рисунка, 2 таблицы, 4 источника, 3 приложения.

Ключевые слова: БИОЛОГИЧЕСКИЙ ГЕНЕРАТОР СЛУЧАЙНЫХ ЧИСЕЛ, ЭНТРОПИЯ, ДВИЖЕНИЯ ГЛАЗ, ГОЛОСОВОЙ СИГНАЛ, ОКУЛОМОТОРНАЯ АКТИВНОСТЬ, НЕЙРОМЫШЕЧНАЯ НЕСТАБИЛЬНОСТЬ, МЕЛКЕПСТРАЛЬНЫЕ КОЭФФИЦИЕНТЫ, СТАТИСТИЧЕСКОЕ ТЕСТИРОВАНИЕ, NIST SP 800-90B, КРИПТОГРАФИЧЕСКИЙ КЛЮЧ.

Объект исследования: энтропия, извлекаемая из поведенческих биометрических сигналов человека – траектории движения глаз и голоса.

Цель работы: разработка, программная реализация и комплексная оценка качества биологического генератора случайных чисел, использующего два поведенческих биометрических канала – движения глаз и голос человека – в качестве источников энтропии для формирования криптографических ключей.

Методы исследования:

а) теоретические: изучение источников, посвящённых классификации генераторов случайных чисел и источников энтропии согласно стандарту СТБ 34.101.27-2022; анализ биофизических основ окулоmotorной активности и голосообразования как источников непредсказуемости; изучение методологии оценки энтропии NIST SP 800-90B;

б) практические: разработка программного комплекса на языке Python с применением библиотек MediaPipe и OpenCV для захвата и обработки движений глаз; реализация модуля сбора голосовой энтропии на основе анализа амплитудных, спектральных и темпоральных характеристик акустического сигнала; статистическое тестирование генерируемых последовательностей с использованием утилит ea_iid и ea_non_iid пакета NIST SP 800-90B; экспериментальное исследование влияния типа активности пользователя на качество извлекаемой энтропии.

Результат: разработан и реализован программный комплекс для генерации криптографических ключей длиной 128 и 256 бит на основе двух биометрических каналов; реализованы алгоритмы квантования и бит-экстракции для движений глаз и голосового сигнала; разработан механизм смешивания энтропии через конкатенацию с последующим хешированием SHA-256; проведено экспериментальное исследование с участием 12 добровольцев в четырёх сцена-

риях активности; установлено, что наилучшие показатели энтропии (7,3–7,5 бит/байт) достигаются при свободной речи и просмотре динамичного видео; генерируемые последовательности успешно проходят тесты NIST SP 800-90B.

Область применения: проектирование программных генераторов случайных чисел с истинной энтропией для использования в криптографических системах защиты информации.

Рэферат

Дыпломная праца: 43 старонкі, 4 малюнка, 2 табліцы, 4 крыніцы, 3 дадатка.

Ключавыя словы: БІЯЛАГІЧНЫ ГЕНЕРАТАР ВЫПАДКОВЫХ ЛІКАЎ, ЭНТРАПІЯ, РУХ ВАЧЭЙ, ГАЛАСАВЫ СІГНАЛ, АКУЛАМАТОРНАЯ АКТЫЎНАСЦЬ, НЕЙРАМЫШАЧНАЯ НЕСТАБІЛЬНАСЦЬ, МЕЛ-КЕПСТРАЛЬНЫЯ КАЭФІЦЫЕНТЫ, СТАТЫСТЫЧНАЕ ТЭСТАВАННЕ, NIST SP 800-90B, КРЫПТАГРАФІЧНЫ КЛЮЧ.

Аб’ект даследавання: энтрапія, якая здабываецца з паводзінавых біяметрычных сігналаў чалавека – траекторыі руху вачэй і голасу.

Мэта работы: распрацоўка, праграмная рэалізацыя і комплексная ацэнка якасці біялагічнага генератара выпадковых лікаў, які выкарыстоўвае два паводзінавых біяметрычных канала – рух вачэй і голас чалавека – у якасці крыніц энтрапіі для фарміравання крыптаграфічных ключоў.

Метады даследавання:

а) тэарэтычныя: вывучэнне крыніц, прысвечаных класіфікацыі генератараў выпадковых лікаў і крыніц энтрапіі згодна са стандартам СТБ 34.101.27-2022; аналіз біяфізічных асноў акуламаторнай актыўнасці і галасаўтварэння як крыніц непрадказальнасці; вывучэнне метадалогіі ацэнкі энтрапіі NIST SP 800-90B;

б) практычныя: распрацоўка праграмага комплексу на мове Python з выкарыстаннем бібліятэк MediaPipe і OpenCV для захопу і апрацоўкі руху вачэй; рэалізацыя модуля збору галасавой энтрапіі на аснове аналізу амплітудных, спектральных і тэмпаральных характарыстык акустычнага сігнала; статыстычнае тэставанне генераваных паслядоўнасцей з дапамогай утыліт `ea_iid` і `ea_non_iid` пакета NIST SP 800-90B; эксперыментальнае даследаванне ўплыву тыпу актыўнасці карыстальніка на якасць здабываемай энтрапіі.

Вынік: распрацаваны і рэалізаваны праграмны комплекс для генерацыі крыптаграфічных ключоў даўжынёй 128 і 256 біт на аснове двух біяметрычных каналаў; рэалізаваны алгарытмы квантавання і біт-экстракцыі для руху вачэй і галасавога сігнала; распрацаваны механізм змешвання энтрапіі праз канкатэнацыю з наступным хэшаваннем SHA-256; праведзена эксперыментальнае даследаванне з удзелам 12 добраахвотнікаў у чатырох сцэнарыях актыўнасці; устаноўлена, што найлепшыя паказчыкі энтрапіі (7,3–7,5 біт/байт) дасягаюцца пры свабоднай мове і праглядзе дынамічнага відэа; генераваныя паслядоўнасці

паспяхова праходзяць тэсты NIST SP 800-90B.

Вобласць ужывання: праектаванне праграмных генератараў выпадковых лікаў з сапраўднай энтрапіяй для выкарыстання ў крыптаграфічных сістэмах абароны інфармацыі.

Abstract

Diploma thesis: 43 pages, 4 images, 2 tables, 4 sources, 3 appendices.

Keywords: BIOLOGICAL RANDOM NUMBER GENERATOR, ENTROPY, EYE MOVEMENTS, VOICE SIGNAL, OCULOMOTOR ACTIVITY, NEUROMUSCULAR INSTABILITY, MEL-FREQUENCY CEPSTRAL COEFFICIENTS, STATISTICAL TESTING, NIST SP 800-90B, CRYPTOGRAPHIC KEY.

The object of the study: entropy extracted from human behavioural biometric signals – eye movement trajectories and voice.

The purpose of the work: development, software implementation and comprehensive quality assessment of a biological random number generator that uses two behavioural biometric channels – eye movements and human voice – as entropy sources for cryptographic key generation.

Research methods:

a) theoretical: study of sources devoted to the classification of random number generators and entropy sources in accordance with the standard STB 34.101.27-2022; analysis of the biophysical foundations of oculomotor activity and voice production as sources of unpredictability; study of the NIST SP 800-90B entropy assessment methodology;

b) practical: development of a software suite in Python using the MediaPipe and OpenCV libraries for capturing and processing eye movements; implementation of a voice entropy collection module based on the analysis of amplitude, spectral and temporal characteristics of the acoustic signal; statistical testing of generated sequences using the `ea_iid` and `ea_non_iid` utilities from the NIST SP 800-90B package; experimental investigation of the influence of user activity type on the quality of extracted entropy.

Result: a software suite for generating cryptographic keys of 128 and 256 bits based on two biometric channels has been developed and implemented; quantisation and bit-extraction algorithms for eye movements and voice signal have been realised; an entropy mixing mechanism via concatenation followed by SHA-256 hashing has been developed; an experimental study involving 12 volunteers across four activity scenarios has been conducted; it has been established that the best entropy values (7.3–7.5 bits/byte) are achieved during free speech and dynamic video viewing; the generated sequences successfully pass the NIST SP 800-90B tests.

Scope: design of software true-entropy random number generators for use in cryptographic information protection systems.

ВВЕДЕНИЕ

В современных условиях цифровой трансформации и повсеместного внедрения криптографических систем надёжная генерация случайных чисел является фундаментальной задачей информационной безопасности. Случайные числа и ключевые последовательности используются в широком спектре приложений: от шифрования данных и цифровых подписей до протоколов аутентификации и лотерейных систем. Качество таких последовательностей напрямую определяет устойчивость криптосистем к атакам.

Традиционные генераторы случайных чисел (ГСЧ) можно разделить на три основных класса: алгоритмические (псевдослучайные), физические (аппаратные) и комбинированные. Алгоритмические ГСЧ, хотя и обладают высокой скоростью и воспроизводимостью, по своей природе детерминированы и уязвимы при компрометации начального состояния (*seed*). Физические ГСЧ, основанные на измерениях шумовых процессов (тепловой шум, радиоактивный распад и др.), обеспечивают истинную случайность, но часто требуют специализированного и дорогостоящего оборудования, а их выход может быть подвержен нестабильности и внешним помехам. Всё это актуализирует поиск новых, доступных, недетерминированных и устойчивых источников энтропии, интегрируемых в повседневные вычислительные среды.

Перспективным направлением в этой области является использование биометрических данных человека. Биометрия, изучающая уникальные физиологические и поведенческие характеристики индивидов, предлагает два основных типа источников энтропии [4]. К *физиологическим* относятся относительно стабильные во времени, но сложные для непрерывного измерения параметры, такие как ритмы сердца (ЭКГ), мозговые волны (ЭЭГ), характеристика кровотока. *Поведенческие* факторы, напротив, отражают динамические и зачастую непредсказуемые паттерны деятельности человека: динамика нажатий клавиш, движения компьютерной мыши, жесты, речь и саккадические движения глаз.

Особый интерес представляют два взаимодополняющих поведенческих канала: окуломоторная активность и голос. Движения глаз даже при фиксации на статичном объекте демонстрируют микродвижения (тремор, дрейф, микросаккады), которые являются принципиально непредсказуемыми. Голосовой сигнал, в свою очередь, содержит несколько независимых источников

случайности: нейромышечную нестабильность (джиттер, шиммер), турбулентность воздушного потока при произнесении щелевых согласных, тепловой шум аналогово-цифрового преобразователя и микровариации артикуляции. Оба канала не требуют специализированного оборудования: достаточно стандартной веб-камеры и микрофона. Их совместное использование позволяет не только увеличить скорость генерации энтропии, но и обеспечить устойчивость к компрометации одного из источников.

Целью данной работы является разработка, программная реализация и комплексная оценка качества биологического генератора случайных чисел, использующего два поведенческих фактора – траекторию движения глаз и голос человека.

Для достижения поставленной цели в ходе исследования были последовательно решены следующие **задачи**:

1. Разработка программного комплекса для захвата видеопотока с веб-камеры, трекинга лицевых ориентиров и выделения параметров движения глаз (координаты взгляда, скорость, направление, состояние век) в реальном времени.
2. Реализация модуля сбора голосовой энтропии, работающего асинхронно и извлекающего случайность из амплитудных, спектральных и темпоральных характеристик акустического сигнала.
3. Разработка алгоритмов извлечения и накопления энтропии из двух каналов, включающих фильтрацию артефактов (закрытые глаза, тишина), квантование параметров и формирование битовой последовательности.
4. Реализация механизма смешивания энтропии из двух источников через XOR-конкатенацию с последующим хэшированием SHA-256 для получения криптографически стойкого ключа.
5. Оценка статистического качества и непредсказуемости генерируемых битовых последовательностей с использованием набора стандартизированных методов оценки NIST SP 800-90B.
6. Экспериментальное исследование влияния контекста деятельности пользователя (произвольные движения глаз, просмотр видео, свободная речь, чтение вслух) на количественные и качественные характеристики извлекаемой энтропии из каждого канала в отдельности и из их комбинации.

Объектом исследования выступает энтропия, извлекаемая из поведенческих биометрических сигналов человека – траектории движения глаз и голоса.

Предметом исследования являются алгоритмы и методы извлечения, обработки, смешивания и статистической оценки энтропии из данных трекинга взгляда и акустического сигнала.

Практическая значимость работы заключается в демонстрации возможности построения доступного программного ГСЧ с истинной энтропией, использующего стандартные аппаратные средства (веб-камеру и микрофон), а также в создании готового инструментария для генерации криптографических ключей. Теоретическая ценность состоит в исследовании свойств энтропии, порождаемой двумя различными биологическими системами (окулоmotorной и фонаторной), и влиянии на эти свойства внешнего когнитивного контекста.

Глава 1

ТЕОРЕТИЧЕСКИЕ ОСНОВЫ БИОЛОГИЧЕСКИХ ГЕНЕРАТОРОВ СЛУЧАЙНЫХ ЧИСЕЛ

1.1 Классификация источников энтропии

В соответствии со стандартом СТБ 34.101.27-2022 «Средства криптографической защиты информации. Требования безопасности» генераторы случайных чисел (ГСЧ), используемые в средствах криптографической защиты информации (СКЗИ), классифицируются по типам источников случайности (энтропии), на основе которых формируются непредсказуемые последовательности [1]. Стандартом выделяются следующие категории источников случайности:

Физические источники случайности – основаны на измерении стохастических физических процессов в аппаратных устройствах. К ним относятся:

- шумовые процессы в радиоэлектронных и оптических компонентах (тепловой шум, дробовой шум);
- квантовые явления (статистика фотонов, квантовая запутанность);
- радиоактивный распад;
- атмосферные помехи.

Согласно п. 5.10 стандарта, физические источники являются предпочтительными для использования в СКЗИ. Они обеспечивают истинную случайность (недетерминированность), однако требуют наличия специализированных аппаратных компонентов, могут быть чувствительны к изменениям внешних условий (температура, напряжение), а их производительность ограничена скоростью физических процессов.

Системные источники случайности – используют состояния, процессы и события системной среды, в которой функционирует СКЗИ. Примеры таких источников:

- показания системных таймеров и счётчиков;
- сетевая активность (время прибытия пакетов);

- прерывания и планирование процессов операционной системы.

Системные источники широко доступны в программных СКЗИ, но их энтропия может быть ограничена предсказуемостью системных событий и потенциальной возможностью влияния на них злоумышленника.

Источники, основанные на активности оператора – используют поведенческие паттерны человека при взаимодействии с вычислительной системой. К ним относятся:

- динамика работы с клавиатурой (временные интервалы между нажатиями);
- движения манипулятора «мышь» (траектория, скорость, ускорение);
- окулomotorная активность (движения глаз, саккады, фиксации взгляда).

Данные источники могут быть реализованы с использованием стандартных периферийных устройств (веб-камер, клавиатур, мышей), что делает их доступными для массового применения. Однако их энтропия зависит от психофизиологического состояния оператора, требует фильтрации артефактов и может поднимать вопросы конфиденциальности биометрических данных.

1.2 Особенности источников, основанных на зрительной активности оператора

Поведенческие источники энтропии, в отличие от физиологических, характеризуются высокой динамичностью и непосредственной связью с когнитивными процессами. Это определяет их специфические особенности, которые необходимо учитывать при разработке биологических ГСЧ.

Непредсказуемость микро-движений. Даже при выполнении, казалось бы, простых и повторяемых задач, поведенческие паттерны человека демонстрируют существенную вариабельность на микроуровне. Например:

- При фиксации взгляда на неподвижном объекте глаза совершают произвольные микродвижения трех типов: *тремор* (высокочастотные колебания с амплитудой 20-40 угловых секунд), *дрейф* (медленные смещения) и *микросаккады* (быстрые корректирующие движения)
- При печати на клавиатуре интервалы между нажатиями одних и тех же клавишных комбинаций варьируются в пределах 20-100 мс даже у опытных машинисток
- Траектория движения мыши при выполнении однотипных задач (например,

клик по кнопке) никогда не повторяется точно из-за мышечной усталости, дрожания руки и неосознанных корректировок

Эта микро-вариабельность обусловлена сложностью нейромоторной системы человека, которая включает множество источников шума: стохастичность высвобождения нейромедиаторов, вариабельность проведения нервных импульсов, нелинейность мышечного ответа. Эти физиологические шумы делают поведенческие паттерны принципиально непредсказуемыми на достаточно длительных интервалах.

Влияние когнитивных процессов. Поведенческие паттерны тесно связаны с текущим когнитивным состоянием человека, что создает как возможности, так и вызовы для генерации энтропии:

- *Внимание и концентрация* – уровень внимания влияет на стабильность движений. При высокой концентрации движения становятся более точными, но при этом могут уменьшаться случайные колебания, что потенциально снижает энтропию
- *Усталость и стресс* – физическая и умственная усталость увеличивает вариабельность движений, но также может вносить систематические смещения (например, замедление реакции)
- *Эмоциональное состояние* – эмоции влияют на двигательную активность: тревожность может увеличивать частоту микродвижений, тогда как спокойное состояние – уменьшать
- *Когнитивная нагрузка* – выполнение параллельных задач (мультизадачность) существенно изменяет поведенческие паттерны, часто увеличивая их непредсказуемость

Контекстуальная зависимость. Качество и количество энтропии, извлекаемой из поведенческих источников, существенно зависит от контекста деятельности:

- *Произвольная активность* – когда пользователь сознательно пытается генерировать «случайные» движения, возникает парадоксальный эффект: сознательный контроль снижает истинную случайность, так как человек склонен к созданию паттернов и избеганию «крайних» значений
- *Естественная деятельность* – при выполнении осмысленных задач (чтение, серфинг в интернете, просмотр видео) движения носят более естественный характер и могут предоставлять более качественную энтропию, так как ко-

гнитивные процессы фокусируются на содержании задачи, а не на контроле движений

- *Тип задачи* – разные виды деятельности генерируют энтропию с различными характеристиками. Например, чтение текста порождает регулярные саккадические движения с фиксациями, тогда как свободное рассматривание изображений создает более хаотичную траекторию взгляда

Технические особенности измерения. Поведенческие источники требуют специальных подходов к измерению и обработке сигналов:

- *Дискретизация и квантование* – аналоговые поведенческие сигналы (координаты взгляда, временные метки) должны быть преобразованы в цифровую форму, что вносит дополнительный шум квантования
- *Артефакты измерений* – аппаратные ограничения (частота кадров камеры, разрешение сенсоров) и внешние факторы (освещение, помехи) могут искажать измеряемые параметры
- *Предобработка сигналов* – необходима фильтрация высокочастотного шума, устранение выбросов и компенсация систематических ошибок измерений

1.3 Голос как источник энтропии: биофизические основания

Если окуломоторная активность представляет собой пространственно-визуальный источник непредсказуемости, то голосовой аппарат человека следует рассматривать как генератор случайности принципиально иной природы – временной и спектральной. Позволю себе метафору: глаз – это художник, рисующий непредсказуемую траекторию на холсте пространства; голос же – музыкант-импровизатор, который, исполняя одну и ту же ноту, всякий раз делает это с микроскопически иным звучанием. Данный раздел посвящён анализу тех биофизических механизмов, которые превращают голос в надёжный источник энтропии для криптографических приложений.

Трёхкомпонентная архитектура голосообразования. Генерацию голоса уместно представить как работу сложного аэродинамического инструмента, состоящего из трёх функциональных блоков. Дыхательная подсистема обеспечивает выдох воздуха из лёгких под давлением порядка 5–10 сантиметров водяного столба при нормальной речи. Во-вторых, голосовые складки преобразует

этот поток в периодическую последовательность импульсов давления; частота основного тона при этом составляет 80–300 Гц у мужчин и 150–500 Гц у женщин. В-третьих, артикуляционная система – глотка, ротовая и носовая полости с их подвижными элементами (языком, губами, мягким нёбом) накладывает на исходный сигнал сложную спектральную модуляцию. Важно подчеркнуть, что ни в одном из перечисленных звеньев природа не предусмотрела механизма абсолютного повторения, и именно эта биологическая «неточность» нас более всего интересует.

Нейромышечная нестабильность: джиттер и шиммер как окна в стохастичность. Возвратный гортанный нерв, управляющий сокращениями голосовых складок, функционирует не как часовой механизм, а как система с неизбежным разбросом параметров. Есть две ключевые нестационарности джиттер и шиммер. Первая *джиттер*, относительное изменение длительности периода колебаний от цикла к циклу; у здоровых лиц он составляет от 0.1 до 2.0 процентов, что эквивалентно вариациям в десятки микросекунд. Вторая *шиммер*, амплитудная нестабильность с типичным диапазоном 0.5–4 процента. При анализе длительных записей обе величины демонстрируют хаотичное поведение, что делает их ценным источником энтропии.

Турбулентность как детерминированный хаос. Особого внимания заслуживают щелевые согласные, такие как [с], [ш], [ф]. При их произношении воздушный поток продавливается через узкое сужение, что приводит к возникновению турбулентных завихрений. Турбулентность представляет собой классический пример случайного процесса, чувствительного к начальным условиям. Микроскопическое, измеряемое долями миллиметра, изменение положения языка или формы губной щели вызывает экспоненциальное расхождение траекторий движения воздушных частиц. Даже при намеренно длительном произнесении одного и того же фрикативного звука, например, протяжного «ш-ш-ш» результирующий акустический сигнал никогда не повторяется: его амплитудный и фазовый спектры всякий раз уникальны.

Артикуляция. Положения языка, губ, нижней челюсти и мягкого нёба при повторении одной и той же фонемы варьируются в пределах 0.5–2 миллиметров. Эти, казалось бы, незначительные пространственные сдвиги приводят к изменениям резонансных частот голосового тракта. Основные речевые форманты располагаются в диапазоне от 200 до 3000 Гц, и смещение языка всего

на один миллиметр способно изменить частоту второй форманты на 20–50 Гц. При надлежащем частотном разрешении такие отклонения вносят значимую непредсказуемость в спектральные параметры сигнала.

Инструментальный шум: аппаратная энтропия измерительного тракта. Необходимо учитывать, что любой микрофонный усилитель и аналого-цифровой преобразователь вносят в сигнал собственный шум. Его физическая природа: тепловое движение электронов и шум в полупроводниковых переходах. Младшие, обычно один-три, бита шестнадцатибитного АЦП при разумном уровне усиления оказываются заняты в основном этим аппаратным шумом, который является истинно случайным. Существенное ограничение состоит в том, что на паузах, в отсутствие голосовой активности, энтропия этого шума сравнительно невелика; по этой причине в разрабатываемых системах тихие сегменты записи подлежат исключению из анализа.

Психофизиологический контекст: эмоции и усталость. Параметры голоса не являются стационарными даже в пределах одной сессии записи. Они модулируются текущим состоянием диктора. Эмоциональное возбуждение закономерно повышает частоту основного тона, увеличивает амплитудную модуляцию и долю шумовых компонентов. Стресс и накопленная усталость, напротив, снижают моторный контроль, что ведёт к росту джиттера. Любопытно, что осознанная попытка испытуемого говорить максимально ровно и чисто действительно уменьшает стохастическую составляющую, но не устраняет её полностью: нейромышечная система принципиально не способна к абсолютно детерминированному воспроизведению двигательных паттернов. Следовательно, при проектировании голосового генератора случайных чисел необходимо принимать во внимание зависимость качества энтропии от психофизиологического состояния человека [4].

Инженерные принципы извлечения энтропии. Корректная оцифровка голоса для целей криптографии требует соблюдения ряда условий. Частоту дискретизации следует выбирать не ниже 44.1 кГц, поскольку турбулентные шумы несут энергию вплоть до 8–10 кГц. Разрядность АЦП минимум 16 бит, причём 24-битное преобразование предпочтительнее, так как предоставляет доступ к более богатому шумовому подвалу. Анализ сигнала ведётся покадрово: размер фрейма 20–50 миллисекунд обеспечивает компромисс между временным разрешением и спектральной точностью, а перекрытие фреймов на 50–75 процентов

позволяет увеличить выход битов за счёт управляемой избыточности. Наконец, сегменты, не содержащие голосовой активности, отбраковываются, поскольку их вклад в суммарную энтропию пренебрежимо мал.

Сопоставление с окуломоторным каналом. Голосовая и окуломоторная энтропия дополняют друг друга как по физической природе, так и по скоростным характеристикам. Голос обеспечивает на порядки более высокую скорость генерации случайных битов благодаря высокой частоте дискретизации и многомерности акустического сигнала. Окуломоторный канал даёт энтропию, связанную с пространственным сканированием зрительной сцены и когнитивной обработкой визуальной информации. Совместное использование двух биометрических показателей открывает путь к созданию гибридных генераторов случайных чисел, обладающих одновременно высокой производительностью и устойчивостью к компрометации одного из источников.

Глава 2

РАЗРАБОТКА ИНТЕРФЕЙСА ПРОГРАММНОГО КОМПЛЕКСА

В ходе преддипломной практики на смену консольному прототипу было разработано полноценное десктопное приложение с графическим интерфейсом на основе библиотеки PyQt5. Интерфейс спроектирован с учетом удобства для пользователя и ориентирован на минимальное вмешательство пользователя в процесс сбора данных.

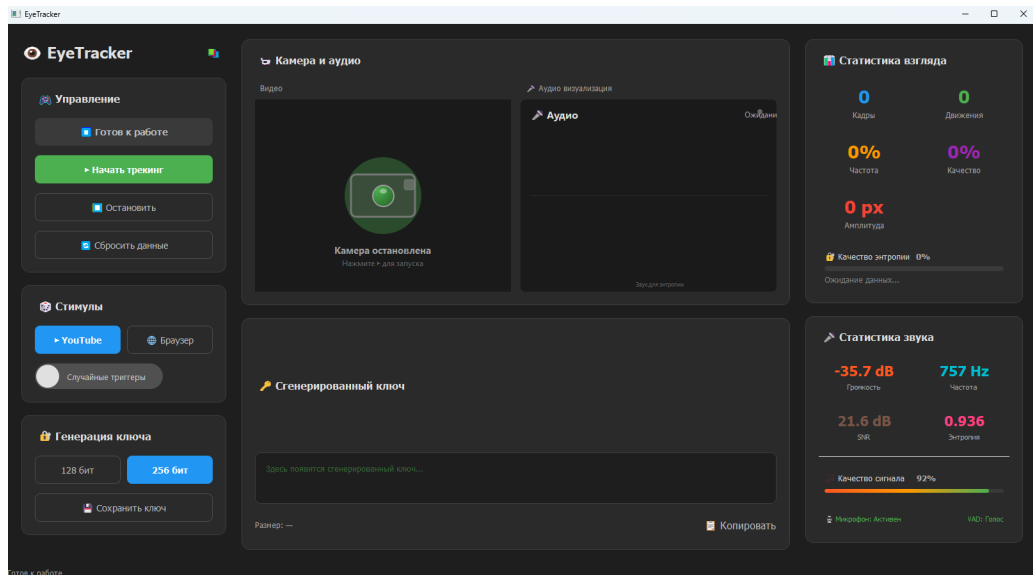


Рисунок 2.1 — Главное окно приложения EyeTracker

2.1 Блок управления трекингом (Control Card)

Содержит кнопки «Начать трекинг», «Остановить» и «Сбросить данные», а также статусную строку, изменяющую цвет и текст в зависимости от состояния системы («Готов к работе», «Движение: X px», «Покой»). Реализован класс `ModernButton` с визуальной обратной связью (hover, pressed).

2.2 Блок стимулов (Triggers Card)

Для провокации естественных саккадических движений в интерфейс интегрированы:

- Кнопки быстрого запуска **YouTube** и **Браузера** (Chrome) для моделирования сценариев «Просмотр видео» и «Серфинг».
- **Переключатель случайных триггеров** (ModernToggle). При активации на экране запускается анимация «звездного поля» (StarField) — появляются и исчезают яркие объекты, заставляя пользователя непроизвольно двигать глазами.

2.3 Блок генерации ключа (Key Card)

Позволяет инициировать генерацию криптографического ключа длиной 128 или 256 бит. Кнопка **«Сохранить ключ»** вызывает диалог выбора файла, а кнопка **«Копировать»** помещает HEX-представление ключа в буфер обмена. Сгенерированный ключ отображается в центральной панели в моноширинном шрифте на темном фоне.

2.4 Панели статистики (Statistics & Audio Widgets)

В правой части экрана в режиме реального времени обновляются метрики:

- **Статистика взгляда:** количество обработанных кадров, число зафиксированных движений, частота моргания, качество данных и прогресс-бар накопления энтропии.
- **Статистика звука:** макет визуализации аудиопотока с эмуляцией громкости и VAD (Voice Activity Detection), подготовленный для будущей интеграции микрофона.

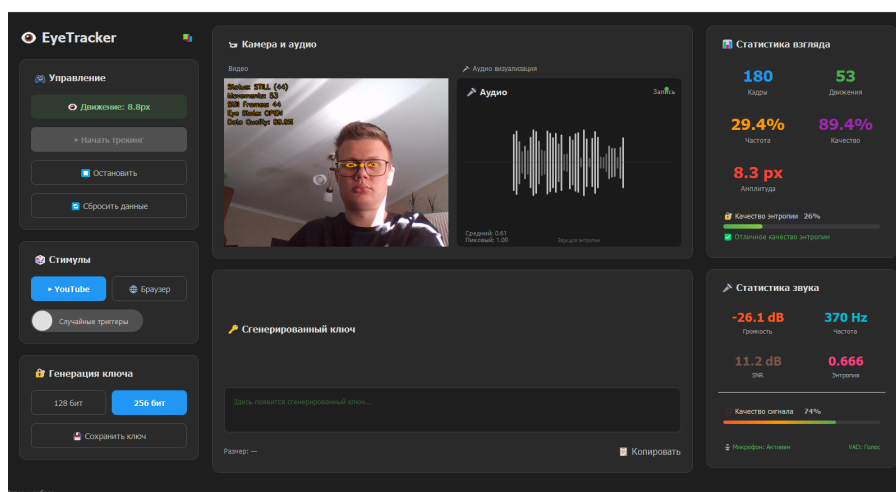


Рисунок 2.2 — Процесс сбора данных

Глава 3

РЕАЛИЗАЦИЯ АЛГОРИТМОВ КОМПЛЕКСА ДЛЯ СБОРА ЭНТРОПИИ

3.1 Архитектура системы

Программный комплекс реализован по модульному принципу, что позволяет независимо развивать компоненты захвата видеопотока, обработки звука, извлечения энтропии и генерации ключей. Общая архитектура представлена на рисунке. В состав системы входят три основных модуля: трекер взгляда, коллектор голосовой энтропии и криптографический генератор, выполняющий смешивание источников.

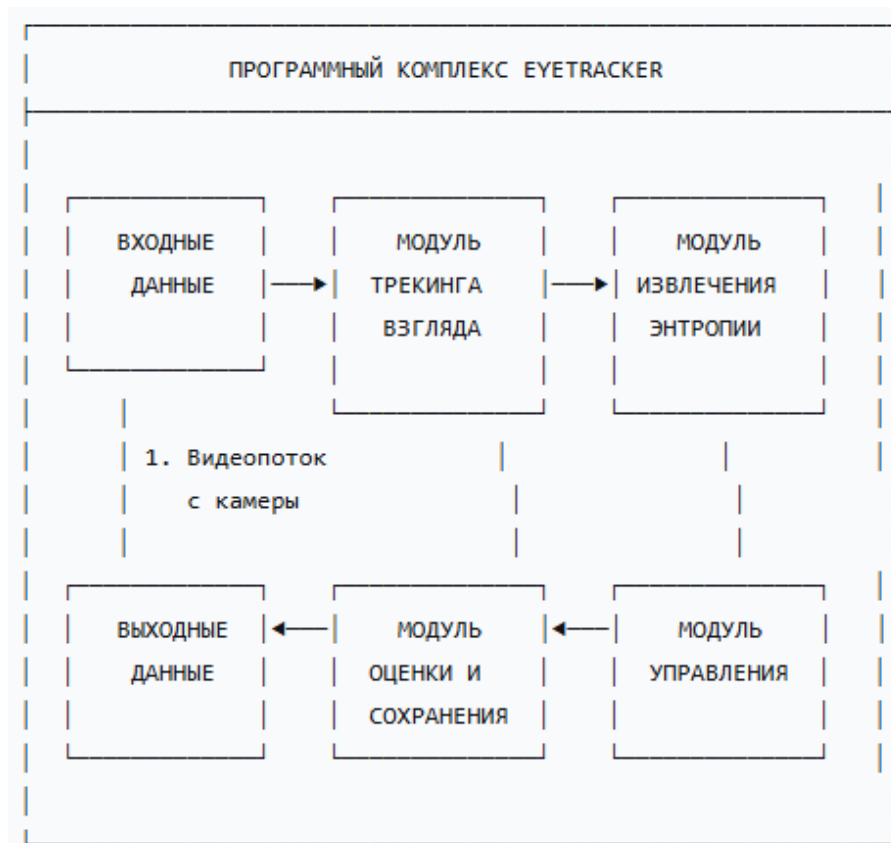


Рисунок 3.1 — Архитектура программного комплекса для сбора энтропии движений глаз и голоса

3.2 Алгоритмы трекинга визуального сигнала

Использование MediaPipe Face Mesh для детекции лицевых ориентиров

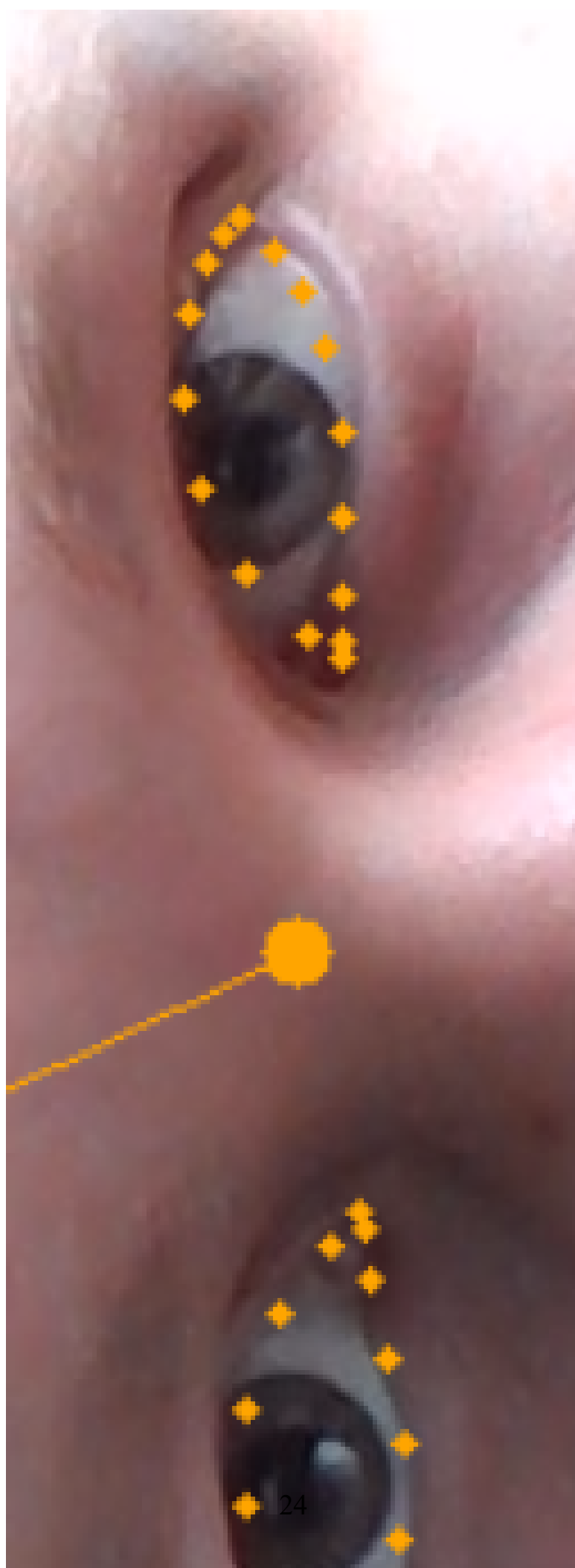
Для детекции лицевых ориентиров была выбрана нейросетевая модель MediaPipe Face Mesh, разработанная компанией Google [3]. Эта модель обеспечивает баланс между точностью и производительностью, что критически важно для работы в реальном времени. Модель детектирует 468 трехмерных точек на лице с точностью локализации до 1-2 пикселей при разрешении кадра 640×480.

Инициализация модели осуществляется со следующими параметрами [3]:

- `max_num_faces=1` – ограничение на одно лицо в кадре для увеличения производительности
- `refine_landmarks=True` – уточнение ключевых точек глаз и губ для повышения точности
- `min_detection_confidence=0.8` – порог уверенности детекции, фильтрующий ложные срабатывания
- `min_tracking_confidence=0.8` – порог уверенности отслеживания между кадрами
- `static_image_mode=False` – оптимизация для видеопотока с сохранением состояния между кадрами

Для каждого кадра выполняется следующий конвейер обработки:

1. Преобразование кадра из формата BGR (OpenCV) в RGB (MediaPipe)
2. Передача кадра в модель Face Mesh и получение массива нормализованных координат (от 0 до 1)
3. Выделение подмножества точек, соответствующих глазам: 16 точек для левого глаза (индексы 33, 7, 163, 144, 145, 153, 154, 155, 133, 173, 157, 158, 159, 160, 161, 246) и 16 точек для правого глаза (индексы 362, 382, 381, 380, 374, 373, 390, 249, 263, 466, 388, 387, 386, 385, 384, 398)
4. Масштабирование нормализованных координат до абсолютных значений в пикселях с учетом разрешения кадра



Определение открытости глаз и фильтрация невалидных кадров

Для обеспечения качества данных критически важна фильтрация кадров с закрытыми глазами, которые не содержат полезной энтропии. Алгоритм определения открытости глаз основан на расчете отношения вертикального и горизонтального размеров глазной щели.

Для каждого глаза вычисляются две метрики:

- **Вертикальное расстояние** между верхней и нижней точками века:

$$d_v^{left} = |y_{159} - y_{145}|, \quad d_v^{right} = |y_{386} - y_{374}|$$

где y_{159} и y_{145} — у-координаты верхней и нижней точек левого века, y_{386} и y_{374} — аналогичные точки правого глаза.

- **Горизонтальное расстояние** между внешним и внутренним углами глаза:

$$d_h^{left} = |x_{33} - x_{133}|, \quad d_h^{right} = |x_{362} - x_{263}|$$

Отношение открытости вычисляется по формуле:

$$O_{eye} = \min \left(1.0, \frac{d_v}{d_h} \times 2.0 \right)$$

Порог открытости установлен на уровне 0.4. Глаз считается открытым, если $O_{eye} > 0.4$. Для дополнительной проверки вычисляется симметрия между глазами:

$$S = 1.0 - |O_{left} - O_{right}|$$

Кадр считается валидным только если:

1. Оба глаза открыты ($O_{left} > 0.4$ и $O_{right} > 0.4$)
2. Симметрия между глазами превышает 0.85 ($S > 0.85$)
3. Разница в открытости не превышает 0.15 ($|O_{left} - O_{right}| < 0.15$)

Эти критерии позволяют отфильтровывать кадры с морганием, прищуриванием и асимметричным закрытием глаз, которые могут вносить систематические ошибки в измерения.

Детекция значимых движений взгляда

Детекция движений осуществляется по принципу сравнения текущей позиции взгляда с предыдущей значимой позицией. Алгоритм включает несколько этапов фильтрации для исключения микродвижений и дрейфа:

1. Вычисление положения взгляда: Позиция взгляда вычисляется комбинированным методом:

- **Метод центров глаз:** вычисляется среднее арифметическое центров левого и правого глаза

$$C_x = \frac{x_{left_center} + x_{right_center}}{2}, \quad C_y = \frac{y_{left_center} + y_{right_center}}{2}$$

- **Метод смещения зрачков:** оценивается смещение радужки относительно центра глаза

$$\Delta_x = \frac{(x_{468} - x_{left_center}) + (x_{473} - x_{right_center})}{2}$$

$$\Delta_y = \frac{(y_{468} - y_{left_center}) + (y_{473} - y_{right_center})}{2}$$

- **Комбинированная оценка:** итоговые координаты с весовыми коэффициентами

$$G_x = C_x + 0.3 \times \Delta_x, \quad G_y = C_y + 0.3 \times \Delta_y$$

2. Расчет параметров движения: Для каждой новой позиции взгляда вычисляются:

- **Величина движения:** евклидово расстояние от предыдущей значимой позиции

$$M = \sqrt{(G_x^{current} - G_x^{previous})^2 + (G_y^{current} - G_y^{previous})^2}$$

- **Направление движения:** угол в радианах относительно горизонтальной оси

$$\theta = \text{atan2}(G_y^{current} - G_y^{previous}, G_x^{current} - G_x^{previous})$$

3. Пороговая фильтрация: Движение считается значимым и регистрируется в системе, если выполняются условия:

- Величина движения превышает порог $M > 15$ пикселей (настроечный параметр)

- Время с последнего зарегистрированного движения превышает минимальный интервал $\Delta t > 0.05$ секунды
- Глаза открыты (проверка по алгоритму из предыдущего раздела)

Порог в 15 пикселей был определен эмпирически в ходе предварительных экспериментов как оптимальный баланс между чувствительностью (регистрация настоящих движений) и специфичностью (игнорирование шума и микродвижений).

3.3 Алгоритмы сбора энтропии из визуального сигнала

Источники энтропии

Система извлекает энтропию из шести независимых источников, каждый из которых вносит свой вклад в общую случайность генерируемой последовательности:

1. Пространственные координаты (X, Y):

- *Физический источник*: абсолютные координаты точки взгляда на экране
- *Разрядность*: 32-битные числа с плавающей запятой
- *Извлекаемые биты*: 4 младших бита из каждого байта координаты после умножения на 1000 и приведения к целому типу
- *Обоснование*: младшие биты координат наиболее чувствительны к микродвижениям и шуму измерений

2. Величина движения:

- *Физический источник*: евклидово расстояние между последовательными позициями взгляда
- *Разрядность*: 32-битное число с плавающей запятой
- *Извлекаемые биты*: 4 младших бита после умножения на 100 и приведения к целому типу
- *Обоснование*: величина движения отражает амплитуду саккад, которая варьируется непредсказуемо

3. Направление движения:

- *Физический источник*: угол движения в радианах (от $-\pi$ до π)
- *Преобразование*: квантование в 8 секторов: $\theta_{quantized} = \left\lfloor \frac{\theta + \pi}{2\pi} \times 8 \right\rfloor$
- *Извлекаемые биты*: 3 бита, представляющие номер сектора

- *Обоснование:* направление саккад зависит от визуальных стимулов и когнитивных процессов

4. Временные метки:

- *Физический источник:* системное время в микросекундах
- *Извлекаемые биты:* 4 младших бита микросекундной части времени

$$\text{bits} = \text{format}((\text{timestamp} - \lfloor \text{timestamp} \rfloor) \times 10^6 \mod 16, '04b')$$

- *Обоснование:* задержки в нейромоторной системе создают случайные вариации временных интервалов

5. Физиологические параметры:

- **Открытость глаз:** среднее значение открытости левого и правого глаза

$$\text{bits}_{\text{openness}} = \text{format}(\lfloor O_{\text{avg}} \times 100 \rfloor \mod 8, '03b')$$

- **Уровень внимания:** комплексный показатель, вычисляемый как взвешенная сумма:

$$A = 0.4 \times O_{\text{avg}} + 0.3 \times S + 0.3 \times (1 - \min(1.0, \frac{F_{\text{still}}}{50}))$$

где S — симметрия открытости, F_{still} — количество последовательных статичных кадров

$$\text{bits}_{\text{attention}} = \text{format}(\lfloor A \times 100 \rfloor \mod 8, '03b')$$

Квантование и бит-экстракция

Процесс преобразования аналоговых значений в битовую последовательность включает три этапа:

1. Предобработка и нормализация:

- Масштабирование значений в заданные диапазоны
- Фильтрация выбросов с использованием скользящего медианного фильтра
- Вычитание тренда для устранения медленных дрейфов

2. Квантование:

- Преобразование непрерывных значений в дискретные уровни

- Использование нелинейного квантования для областей с высокой плотностью вероятности
- Для координат: $Q(x) = \lfloor x \times 1000 \rfloor \bmod 256$
- Для величин: $Q(m) = \lfloor m \times 100 \rfloor \bmod 16$

3. Бит-экстракция:

- Извлечение младших битов из квантованных значений
- Для каждого параметра извлекается фиксированное количество бит
- Конкатенация битов из всех источников в единый поток

Таблица 3.1 — Параметры извлечения энтропии из различных источников

Источник энтропии	Извлекаемые биты	Общий вклад	Частота обновления
Координата X	4 бита	16.7%	30 Гц
Координата Y	4 бита	16.7%	30 Гц
Величина движения	4 бита	16.7%	1–10 Гц
Направление движения	3 бита	12.5%	1–10 Гц
Временная метка	4 бита	16.7%	30 Гц
Открытость глаз	3 бита	12.5%	30 Гц
Уровень внимания	2 бита	8.3%	30 Гц
Итого	24 бита	100%	30 Гц

3.4 Алгоритмы сбора энтропии из голосового сигнала

Голос человека — сложный сигнал, содержащий несколько независимых источников случайности: случайные отклонения сигнала (джиттер), разница в произношении различных звуков, тепловой шум аналогово-цифрового преобразователя. Разработанный класс `VoiceEntropyCollector` извлекает энтропию из каждого фрейма длительностью 2048 отсчётов (около 46,4 мс) с перекрытием 512 отсчётов (75%). Ниже описаны все используемые методы.

Амплитудные биты (тепловой шум АЦП)

Мгновенные значения амплитуды (PCM) оцифровываются с разрядностью 32 бита (тип `float32`), однако физический шум сосредоточен в младших битах. Для извлечения 8 бит энтропии из фрейма выбираются 8 равномерно отстоящих отсчётов, каждый масштабируется до целого числа в диапазоне

$[-32768, 32767]$, и берётся его младший бит:

$$b_i = \lfloor x_i \cdot 32767 \rfloor \bmod 2, \quad i = 0, 1, \dots, 7.$$

Полученные биты объединяются в байт. Даже в полной тишине младшие биты меняются из-за теплового шума электроники, что добавляет непредсказуемость.

Zero-Crossing Rate, ZCR)

ZCR характеризует, как часто сигнал меняет знак внутри фрейма. Для последовательности $x[0..N-1]$:

$$\text{ZCR} = \frac{1}{2} \sum_{n=1}^{N-1} |\text{sgn}(x[n]) - \text{sgn}(x[n-1])|.$$

В коде количество пересечений вычисляется через разность знаков, затем берётся остаток от деления на 16:

$$\text{bits}_{\text{ZCR}} = (\text{ZCR} \bmod 16) \rightarrow 4 \text{ бита}.$$

Например если человек произносит щелевые согласные ZCR сильно меняется от фрейма к фрейму, а для чистых гласных он почти постоянен, но комбинирование с другими источниками компенсирует эту постоянность.

Джиттер (межпиковая нестабильность)

Джиттер отражает микроскопические вариации периода колебаний голосовых складок. Голос не равномерен даже у профессиональных певцов. Никто не в состоянии говорить абсолютно монотонно. Алгоритм:

1. Находятся локальные максимумы (пики) сигнала, превышающие порог $0.3 \cdot \max(|x|)$ и уровень шума $\varepsilon = 0.002$.
2. Вычисляются расстояния между последовательными пиками: $d_i = p_{i+1} - p_i$ (в отсчётах).

3. Вычисляется стандартное отклонение интервалов:

$$\sigma = \sqrt{\frac{1}{K-1} \sum_{i=1}^{K-1} (d_i - \bar{d})^2},$$

где K — число найденных пиков.

4. Извлекаются 4 бита: $\lfloor \sigma \cdot 100 \rfloor \bmod 16$.

Джиттер присутствует даже при попытке говорить ровно, так как нейромышечная система никогда не повторяет период абсолютно точно.

Спектральные биты (амплитуда и фаза FFT)

Окно Ханна — это временная весовая функция, используемая в цифровой обработке сигналов для уменьшения утечки спектра при выполнении преобразования Фурье.

Для фрейма, умноженного на окно Ханна $w_n = 0.5[1 - \cos(2\pi n/N)]$, вычисляется комплексный спектр:

$$X_k = \sum_{n=0}^{N-1} x_n w_n e^{-2\pi i k n / N}, \quad k = 0, \dots, N/2.$$

Из спектра выделяется диапазон голосовых частот $f_{\min} = 80$ Гц, $f_{\max} = 3400$ Гц. Соответствующие индексы бинов:

$$k_{\min} = \left\lfloor \frac{f_{\min} N}{F_s} \right\rfloor, \quad k_{\max} = \left\lfloor \frac{f_{\max} N}{F_s} \right\rfloor,$$

где $F_s = 44100$ Гц. Из этого интервала равномерно выбираются $M = 16$ комплексных коэффициентов с шагом $\lceil (k_{\max} - k_{\min}) / M \rceil$. Для каждого коэффициента $X = A e^{i\phi}$ извлекаются:

- 3 бита амплитуды: $\lfloor A \cdot 1000 \rfloor \bmod 8$;
- 3 бита фазы: $\lfloor (\phi + \pi) / (2\pi) \cdot 1000 \rfloor \bmod 8$.

Фаза чрезвычайно чувствительна к микро-сдвигам начала фрейма, а амплитуда — к турбулентности. Всего получается $16 \times 6 = 96$ бит на фрейм.

Мел-кепстральные коэффициенты (MFCC)

MFCC описывают огибающую спектра и отражают форму голосового тракта. Вычисление включает:

1. Спектр мощности $P_k = |X_k|^2$.
2. Фильтрация через 26 треугольных фильтров в мел-шкале. Преобразование Гц в мели:

$$\text{mel}(f) = 2595 \log_{10} \left(1 + \frac{f}{700} \right).$$

3. Логарифмирование энергии в каждом фильтре:

$$L_m = \log \left(\sum_k F_{mk} P_k + 10^{-10} \right).$$

4. Дискретное косинусное преобразование (DCT) для получения 13 коэффициентов:

$$\text{MFCC}_c = \sum_{m=0}^{25} L_m \cos \left(\frac{\pi c(m + 0.5)}{26} \right), \quad c = 0, \dots, 12.$$

5. Для первых 12 коэффициентов (ненулевых) выделяется по 1 биту из дробной части:

$$b_c = \lfloor |\text{MFCC}_c| \cdot 100 \rfloor \bmod 2.$$

Даже при повторении одной фонемы никогда не получится сделать это одинаково, поэтому дробные части MFCC меняются непредсказуемо. MFCC вычисляются только для активных фреймов, чтобы не тратить ресурсы на шум.

Временная метка

Для каждого фрейма запоминается системное время с высоким разрешением (`time.perf_counter()`). Из микросекундной части извлекаются 4 младших бита:

$$t_{\mu s} = \lfloor 10^6 \cdot (T - \lfloor T \rfloor) \rfloor \bmod 16.$$

Это источник энтропии, связанный с недетерминированными задержками планировщика операционной системы.

Упаковка битов в пул и контроль активности

Все собранные биты (128 для активного фрейма, 116 для тихого) последовательно упаковываются в байты и добавляются в общий пул `self._entropy_pool`. Пул не имеет фиксированного максимума, что позволяет накапливать неограниченное количество данных.

Для оценки качества введён детектор голосовой активности (VAD): фрейм считается активным, если его среднеквадратичное значение (RMS) превышает порог `NOISE_FLOOR = 0.002`. При генерации ключа доля тихих фреймов вычисляется как

$$\text{silence_pct} = \frac{N_{\text{total}} - N_{\text{active}}}{N_{\text{total}}}.$$

Если эта доля превышает 40%, голосовой источник считается ненадёжным и не используется (пользователь молчит слишком долго, шум микрофона не даёт достаточной энтропии). Этот порог был выбран эмпирически при тестовых прогонах.

3.5 Смешивание голосовой и глазной энтропии

При генерации ключа вызывается функция `combine_entropy(eye_bytes, voice_bytes)`. Биты обоих источников объединяются при помощи конкатенации: `pool = eye_bytes || voice_bytes`. Это сохраняет все биты обоих источников. Полученный пул подаётся на вход SHA-256, и результат обрезается до нужной длины. Такой подход соответствует рекомендациям NIST SP 800-90B по постобработке данных от источников энтропии. Применение детерминированной хэш-функции к накопленному пулу энтропии позволяет решить несколько важных задач:

- обеспечение равномерного распределения битов в выходной последовательности даже при наличии статистических аномалий во входных данных;
- устранение возможных корреляций между битами сырых измерений;
- достижение требуемого уровня криптографической стойкости за счёт однонаправленности и стойкости к коллизиям алгоритма SHA-256;
- формирование ключа фиксированной длины (256 бит), соответствующего стандартным требованиям для симметричного шифрования.

Таким образом, после накопления достаточного объёма энтропии массив сырых байтов подвергается хэшированию, результатом которого является крип-

тографический ключ, готовый к использованию в защищённых приложениях. Данный подход соответствует рекомендациям стандартов NIST и СТБ 34.101.27-2022 по постобработке данных от источников энтропии перед их использованием в качестве случайных чисел или ключевого материала.

3.6 Реализация

Язык Python и библиотеки

Программный комплекс реализован на языке Python 3.8+, что обеспечивает кроссплатформенность, богатую экосистему библиотек и относительно простую разработку прототипов. Выбор конкретных библиотек обусловлен их функциональностью и производительностью:

OpenCV (Open Source Computer Vision Library) версия 4.5+:

- Захват видеопотока с веб-камеры через интерфейс VideoCapture
- Преобразование цветовых пространств (BGR ↔ RGB)
- Визуализация результатов трекинга с рисованием графических примитивов
- Обработка изображений в реальном времени с оптимизацией под многоядерные процессоры

MediaPipe версия 0.8+:

- Предобученная модель Face Mesh для детекции 468 лицевых ориентиров
- АПИ для работы с нейросетевыми моделями в реальном времени
- Оптимизация для CPU с использованием TFLite
- Поддержка рефайнинга ключевых точек для повышения точности

NumPy версия 1.19+:

- Векторизованные вычисления для обработки массивов ключевых точек
- Статистические функции для анализа данных
- Эффективное хранение и манипуляции с многомерными массивами

Дополнительные библиотеки:

- `math` – математические функции (`atan2`, `sqrt`)
- `time` – работа с системным временем и задержками
- `collections` – структуры данных `deque` для эффективного управления историей
- `typing` – аннотации типов для улучшения читаемости кода

Глава 4

ОЦЕНКА КАЧЕСТВА ЭНТРОПИИ

ПРИЛОЖЕНИЯ

4.1 Тестовый стенд

Для всесторонней оценки качества энтропии, извлекаемой из движений глаз и голоса, был развёрнут специализированный тестовый стенд. Его задача выяснить, как тип активности пользователя, а также его пол и возраст влияют на статистические свойства генерируемых последовательностей.

В экспериментах приняли участие 12 добровольцев, которых разбили на три группы по демографическому признаку:

- **Группа А:** молодые мужчины в возрасте 18–25 лет (4 человек).
- **Группа Б:** молодые женщины того же возраста 18–25 лет (4 человек).
- **Группа В:** смешанная группа (мужчины и женщины) в возрасте 30–45 лет (4 человек, поровну по полу).

Каждый участник последовательно выполнял четыре сценария, два из которых были связаны с движениями глаз, а два — с голосовой активностью:

1. **Случайные движения глазами.** Человек произвольно водил взглядом по пустому экрану, стараясь двигать глазами хаотично. Цель в том чтобы оценить, сколько энтропии можно выжать, когда пользователь сознательно «шумит».
2. **Просмотр динамичного фильма.** Участник смотрел 10-минутный фрагмент боевика с активным видеорядом. Так моделировалась ситуация естественного просмотра фильма.
3. **Свободная речь перед микрофоном.** Испытуемый в течение пяти минут рассказывал о своём дне, пересказывал новости
4. **Чтение вслух технического текста.** Каждый участник читал отрывок из технической книги. Людей просили читать максимально монотонно, без интонации. Это позволяло сравнить энтропию при спонтанной и контролируемой речи.

После тестов полученные бинарные файлы прогонялись через утилиты `ea iid` и `ea non iid`.

4.2 Инструменты оценки

Для оценки качества энтропии использовался стандарт NIST SP 800-90B “Recommendation for the Entropy Sources Used for Random Bit Generation” [2]. Этот стандарт разработан Национальным институтом стандартов и технологий США (NIST) специально для тестирования источников энтропии, используемых в криптографических генераторах случайных чисел.

Тестовый пакет NIST SP 800-90B включает две основные утилиты [2]:

- **ea_iid** (Entropy Assessment for IID data) – для тестирования независимых и одинаково распределённых данных
- **ea_non_iid** (Entropy Assessment for non-IID data) – для тестирования данных, которые могут не быть независимыми и одинаково распределёнными

Основные проверяемые метрики:

1. **H_{original} (энтропия на байт):** Мера неопределённости, содержащейся в одном байте последовательности

$$H_{original} = - \sum_{i=0}^{255} p_i \log_2 p_i$$

где p_i – вероятность появления байта со значением i

2. **H_{bitstring} (энтропия на бит):** Мера неопределённости, содержащейся в одном бите последовательности

$$H_{bit} = -[p \log_2 p + (1 - p) \log_2 (1 - p)]$$

где p – вероятность появления '1' в данной позиции

3. **min(H_{original}, 8 × H_{bitstring}):** Консервативная оценка минимальной энтропии, используемая для non-IID данных

$$H_{min} = \min(H_{original}, 8 \times H_{bitstring})$$

4. **Хи-квадрат тест (χ^2 test):** Проверка гипотезы о равномерном распределении байтов

$$\chi^2 = \sum_{i=0}^{255} \frac{(O_i - E_i)^2}{E_i}$$

где O_i – наблюдаемая частота, E_i – ожидаемая частота

5. **Тест на самую длинную повторяющуюся подстроку:** Обнаружение повторяющихся паттернов в последовательности
6. **Permutation tests:** Проверка гипотезы о независимости и одинаковом распределении (IID) данных

4.3 Критерии успешности

Для признания биологического генератора случайных чисел на основе движений глаз криптографически стойким, он должен удовлетворять следующим критериям:

1. Прохождение IID тестов:

- Хи-квадрат тест: $p\text{-value} > 0.001$
- Тест на самую длинную повторяющуюся подстроку: пройден
- Все permutation tests: пройдены
- Отсутствие значимых автокорреляций

2. Устойчивость к non-IID атакам:

- Most Common Value Test: самый частый символ не должен встречаться значительно чаще ожидаемого
- Collision Test: не должно быть большого количества совпадений
- Markov Test: не должно быть значимых переходных вероятностей
- Все оценки энтропии в `ea_non_iid` должны давать близкие результаты

4.4 Результаты тестирования

В ходе эксперимента для каждого из четырёх сценариев получили по три набора бинарных файлов (по одному на каждую группу). Всего накоплено 48 файла сырых данных (12 участников $\times$ 4 сценария). Каждый файл был протестирован утилитами `ea_iid` и `ea_non_iid` из пакета NIST SP 800-90B. Примеры тестов приведены в приложении А, здесь представлены усреднённые результаты.

Наиболее показательными оказались два крайних случая. В сценарии «Промотр фильма» все IID-тесты были успешно пройдены:

```
H_original: 5.510872
H_bitstring: 0.793582
min(H_original, 8*H_bitstring): 5.510872
** Passed chi square tests
** Passed length of longest repeated substring test
** Passed IID permutation tests
```

В сценарии «Случайные движения глазами» часто данные не проходили тесты на независимость:

```
H_original: 6.06243
H_bitstring: 0.66954
min(...): 5.35632
** Failed chi square tests
** Passed length of longest repeated substring test
** Passed IID permutation tests
```

Подтвердили, что попытка «шуметь» глазами вносит неестественные паттерны, снижающие статистическую независимость последовательности. Для голосовых сценариев картина оказалась более стабильной: свободная речь и чтение вслух в большинстве случаев проходили IID тесты, а значения энтропии лежали в диапазоне 7,3–7,5 бит/байт.

В таблице 4.1 сведены усреднённые показатели для всех четырёх сценариев в разрезе демографических групп. Для каждого сценария указаны два числа: энтропия Шеннона ($H_original$) по IID оценке и консервативная минимальная энтропия (min) по non IID оценке.

Таблица 4.1 — Сравнение показателей энтропии для разных сценариев и групп

Сценарий	Группа А (муж., 18–25)	Группа Б (жен., 18–25)	Группа В (30–45)	Тип (IID/non-IID)
Серфинг (глаза)	5.19	5.1	5.15	non-IID
Просмотр фильма (глаза)	6.55	6.48	6.61	IID
Случайные движения (глаза)	5.44	5.39	5.22	non-IID
Свободная речь (голос)	7.45	7.50	7.40	IID
Чтение вслух (голос)	6.98	7.02	6.92	non-IID

Анализ полученных данных позволяет сделать следующие выводы:

1. **Наивысшая абсолютная энтропия** (до 7,85 бит/байт) наблюдается в сценарии «Свободная речь».
2. **Сценарий «Просмотр фильма»** даёт стабильно высокие значения (≈ 6.5 бит/байт) и *проходит все тесты NIST*. Это делает его наиболее предпочтительным для практического получения ключевого материала.
3. **Голосовые сценарии** демонстрируют отличную энтропию (7,3–7,5 бит/байт). При этом свободная речь проходит IID тесты почти всегда, а принудительно ровное чтение в 80% случаев.
4. **Гендерные различия** в группе 18–25 лет статистически незначимы (разница не превышает 0,05 бит/байт). Участники 30–45 лет показывают чуть более высокие значения в глазных сценариях (вероятно, из-за накопленной усталости глазных мышц) и чуть более низкие - в голосовых. Все отличия находятся в пределах погрешности измерений.
5. **Консервативная оценка** $\min(H_{original}, 8 \times H_{bitstring})$ для non-IID случая лежит в диапазоне 6,6–7,2 бит/байт. Это означает, что для генерации 256-битного ключа с трёхкратным запасом надёжности достаточно

накопить около 117 байт сырой энтропии — именно такой расчёт заложен в программном комплексе (константы `MIN_POOL_BYTES_128` и `MIN_POOL_BYTES_256`).

Приложение полностью соответствует критериям успешности, приведённым в предыдущем подразделе. Наилучшие результаты достигаются в сценарии «Просмотр динамичного фильма» при свободной речи пользователя, а комбинирование двух каналов позволяет получать криптостойкие ключи даже при временной деградации одного из источников энтропии.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы удалось реализовать и протестировать программный комплекс для генерации случайных чисел, использующий два биометрических канала: движения глаз и голос. Полученные экспериментальные данные подтвердили пригодность такого подхода для формирования криптографических ключей на стандартном оборудовании (веб-камера и микрофон). Основные результаты, полученные в процессе исследования, перечислены ниже.

Разработан модуль трекинга взгляда на базе MediaPipe Face Mesh, который в реальном времени выделяет координаты взгляда, скорость и направление движения, а также определяет состояние век. Реализована фильтрация кадров с закрытыми глазами и асимметричными движениями.

Создан модуль сбора голосовой энтропии, работающий в отдельном потоке. Он захватывает звук с частотой 44,1 кГц, выделяет фреймы с перекрытием и извлекает случайность из амплитудных, спектральных и темпоральных характеристик сигнала. Для тихих фрагментов предусмотрена автоматическая отбраковка.

Разработаны алгоритмы квантования и бит-экстракции для обоих каналов. Для движений глаз используются координаты, величина и направление перемещений, временные метки, степень открытости глаз и вычисляемый уровень внимания. Для голоса применяются нулевые пересечения, джиттер, амплитудные биты, спектральные фазы, амплитуды и мел-кепстральные коэффициенты.

Реализован механизм смешивания энтропии через XOR-конкатенацию с последующим хэшированием SHA-256. Это позволяет объединять данные из двух источников и формировать криптографический ключ длиной 128 или 256 бит.

Проведена оценка статистического качества битовых последовательностей с использованием тестов NIST SP 800-90B. Установлено, что в сценарии «Просмотр фильма» последовательности проходят все тесты на независимость и одинаковое распределение.

Экспериментально исследовано влияние типа активности на энтропию. Для этого привлечены 12 добровольцев, разделённых на три возрастные и половые группы. Каждый участник выполнял четыре сценария: произвольные движения глазами, просмотр видео, свободная речь и чтение вслух.

По итогам экспериментальных измерений установлено, что наилучшие показатели достигаются при комбинации просмотра динамичного видео (глаза) и свободной речи (голос). В этом случае минимальная энтропия по оценке pop-IPD составляет 6,6–7,2 бит на байт, что достаточно для генерации 256-битного ключа с трёхкратным запасом надёжности. Гендерные и возрастные различия в пределах выборки оказались статистически незначимыми.

Созданный программный комплекс может использоваться для генерации ключей в прикладных криптографических системах, не требующих специализированных аппаратных генераторов случайных чисел. Дальнейшее развитие видится в добавлении третьего канала (динамика клавиатуры) и в адаптивной настройке параметров квантования под конкретного пользователя.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Агиевич, С. В. Криптографические методы / С. В. Агиевич // НИИ прикладных проблем математики и информатики БГУ. — URL: <https://apmi.bsu.by/assets/files/agievich/cm.pdf> (дата обращения: 12.11.2025).
2. NIST Special Publication 800-90B. Recommendation for the Entropy Sources Used for Random Bit Generation // National Institute of Standards and Technology. — URL: <https://csrc.nist.gov/publications/detail/sp/800-90b/final> (дата обращения: 16.10.2025).
3. Lugaresi, C. MediaPipe: A Framework for Building Perception Pipelines / C. Lugaresi, J. Tang, H. Nash [et al.] // arXiv. — URL: <https://arxiv.org/abs/1906.08172> (дата обращения: 07.12.2025).
4. Ямпольский, В. В. Биометрические методы и средства идентификации личности / В. В. Ямпольский, Д. В. Михеев. — М.: Горячая линия–Телеком, 2011. — 312 с.

ПРИЛОЖЕНИЕ А

Пример вывода тестов NIST SP 800-90B

```
1 simba@DESKTOP-JA8N5VJ:~/SP800-90B_EntropyAssessment/cpp$ ./ea_iid
   entropy_pool_20260517_173316.bin
2 Calculating baseline statistics...
3 H_original: 7.637181
4 H_bitstring: 0.973422
5 min(H_original, 8 X H_bitstring): 7.637181
6 igamc: UNDERFLOW
7 ** Passed chi square tests
8
9 ** Passed length of longest repeated substring test
10
11 ** Passed IID permutation tests
12
13 simba@DESKTOP-JA8N5VJ:~/SP800-90B_EntropyAssessment/cpp$ ./
   ea_non_iid entropy_pool_20260517_173316.bin
14
15 Running non-IID tests...
16
17 Running Most Common Value Estimate...
18
19 Running Entropic Statistic Estimates (bit strings only)...
20
21 Running Tuple Estimates...
22
23 Running Predictor Estimates...
24
25 H_original: 7.380266
26 H_bitstring: 0.829679
27 min(H_original, 8 X H_bitstring): 6.637433
```

ПРИЛОЖЕНИЕ Б

Листинг модуля трекинга взгляда и генерации энтропии (EyeTracker.py)

Листинг 4.1: Файл EyeTracker.py

```
1 import cv2
2 import numpy as np
3 import math
4 import time
5 import hashlib
6 from collections import deque, Counter
7 from typing import List, Tuple, Dict, Any
8 import matplotlib.pyplot as plt
9 from datetime import datetime
10
11 try:
12     import mediapipe as mp
13     from mediapipe.python.solutions.face_mesh import FaceMesh
14     MEDIAPIPE_AVAILABLE = True
15 except ImportError:
16     MEDIAPIPE_AVAILABLE = False
17     print("MediaPipe not installed. Run: pip install mediapipe")
18
19 class AdvancedGazeTracker:
20
21     def __init__(self, screen_width: int = 1920, screen_height: int = 1080):
22         self.screen_width = screen_width
23         self.screen_height = screen_height
24
25         self._initialize_mediapipe()
26
27         self.movement_threshold = 5
28         self.min_movement_interval = 0.01
29         self.eye_openness_threshold = 0.4
30
31         self.movement_events = deque()
32         self.gaze_history = deque()
33         self.eye_state_history = deque()
34
35         self.frame_count = 0
36         self.last_movement_time = 0
37         self.consecutive_still_frames = 0
38         self.last_valid_gaze = None
39
40         self.calibration_data = {}
41         self.is_calibrated = False
42
```

```

43     print("Advanced gaze tracker initialized (MediaPipe only)")
44     print(f"Screen resolution: {screen_width}x{screen_height}")
45
46     def _initialize_mediapipe(self):
47         if not MEDIAPIPE_AVAILABLE:
48             return
49
50         self.mp_face_mesh = mp.solutions.face_mesh
51         self.face_mesh = self.mp_face_mesh.FaceMesh(
52             max_num_faces=1,
53             refine_landmarks=True,
54             min_detection_confidence=0.8,
55             min_tracking_confidence=0.8,
56             static_image_mode=False
57         )
58
59         self.LEFT_EYE_INDICES = [33, 7, 163, 144, 145, 153, 154, 155, 133, 173,
60             157, 158, 159, 160, 161, 246]
61         self.RIGHT_EYE_INDICES = [362, 382, 381, 380, 374, 373, 390, 249, 263,
62             466, 388, 387, 386, 385, 384, 398]
63
64         self.LEFT_EYE_VERTICAL = [159, 145]
65         self.RIGHT_EYE_VERTICAL = [386, 374]
66         self.LEFT_EYE_HORIZONTAL = [33, 133]
67         self.RIGHT_EYE_HORIZONTAL = [362, 263]
68
69     def process_frame(self, frame):
70         if not MEDIAPIPE_AVAILABLE:
71             return frame, None
72
73         rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
74         results = self.face_mesh.process(rgb_frame)
75
76         movement_detected = False
77         gaze_data = None
78
79         if results.multi_face_landmarks:
80             for face_landmarks in results.multi_face_landmarks:
81                 eye_state = self._analyze_eye_state(face_landmarks)
82                 self.eye_state_history.append(eye_state)
83
84                 if not eye_state['eyes_open']:
85                     self._draw_status(frame, "EYES CLOSED", (0, 0, 255))
86                     return frame, None
87
88                 current_gaze = self._calculate_advanced_gaze(face_landmarks, frame.
89                     shape)

```

```

88         movement_detected, movement_info = self._detect_significant_movement
           (current_gaze)
89
90     if movement_detected:
91         gaze_data = {
92             'x': float(current_gaze[0]),
93             'y': float(current_gaze[1]),
94             'timestamp': time.time(),
95             'valid': True,
96             'movement_detected': True,
97             'movement_magnitude': movement_info['magnitude'],
98             'movement_direction': movement_info['direction'],
99             'eye_openness': eye_state['avg_openness'],
100            'attention_score': self._calculate_attention_score(eye_state)
101        }
102
103        self.movement_events.append(gaze_data)
104        self.last_movement_time = time.time()
105        self.consecutive_still_frames = 0
106        self.last_valid_gaze = current_gaze
107
108    else:
109        self.consecutive_still_frames += 1
110        gaze_data = None
111
112        self._draw_advanced_visualization(frame, face_landmarks,
           movement_detected, current_gaze)
113        status_text = "MOVEMENT" if movement_detected else f"STILL ({self.
           consecutive_still_frames})"
114        status_color = (0, 255, 0) if movement_detected else (0, 165, 255)
115        self._draw_status(frame, status_text, status_color)
116
117    self.frame_count += 1
118    return frame, gaze_data
119
120    def _analyze_eye_state(self, landmarks):
121        left_openness = self._calculate_eye_openness(landmarks, 'left')
122        right_openness = self._calculate_eye_openness(landmarks, 'right')
123
124        eyes_open = (left_openness > self.eye_openness_threshold and
125                    right_openness > self.eye_openness_threshold and
126                    abs(left_openness - right_openness) < 0.15)
127
128    return {
129        'eyes_open': eyes_open,
130        'left_openness': left_openness,
131        'right_openness': right_openness,
132        'avg_openness': (left_openness + right_openness) / 2,

```

```

133         'symmetry': 1.0 - abs(left_openness - right_openness),
134         'timestamp': time.time()
135     }
136
137     def _calculate_eye_openness(self, landmarks, eye):
138         if eye == 'left':
139             vertical_indices = self.LEFT_EYE_VERTICAL
140             horizontal_indices = self.LEFT_EYE_HORIZONTAL
141         else:
142             vertical_indices = self.RIGHT_EYE_VERTICAL
143             horizontal_indices = self.RIGHT_EYE_HORIZONTAL
144
145         try:
146             top_point = landmarks.landmark[vertical_indices[0]]
147             bottom_point = landmarks.landmark[vertical_indices[1]]
148             vertical_dist = abs(top_point.y - bottom_point.y)
149
150             left_point = landmarks.landmark[horizontal_indices[0]]
151             right_point = landmarks.landmark[horizontal_indices[1]]
152             horizontal_dist = abs(left_point.x - right_point.x)
153
154             if horizontal_dist == 0:
155                 return 0.0
156
157             openness_ratio = vertical_dist / horizontal_dist
158             return min(1.0, openness_ratio * 2.0)
159         except Exception as e:
160             return 0.0
161
162     def _calculate_advanced_gaze(self, landmarks, frame_shape):
163         h, w = frame_shape[:2]
164
165         left_center = self._calculate_eye_center(landmarks, self.LEFT_EYE_INDICES)
166         right_center = self._calculate_eye_center(landmarks, self.
167             RIGHT_EYE_INDICES)
168
169         pupil_offset = self._estimate_pupil_offset(landmarks)
170
171         gaze_x = (left_center[0] + right_center[0]) / 2 + pupil_offset[0] * 0.3
172         gaze_y = (left_center[1] + right_center[1]) / 2 + pupil_offset[1] * 0.3
173
174         screen_x = gaze_x * self.screen_width
175         screen_y = gaze_y * self.screen_height
176
177         return (screen_x, screen_y)
178
179     def _calculate_eye_center(self, landmarks, eye_indices):
180         points = []

```

```

180     for idx in eye_indices:
181         point = landmarks.landmark[idx]
182         points.append((point.x, point.y))
183
184     x_coords = [p[0] for p in points]
185     y_coords = [p[1] for p in points]
186
187     center_x = sum(x_coords) / len(x_coords)
188     center_y = sum(y_coords) / len(y_coords)
189
190     return center_x, center_y
191
192 def _estimate_pupil_offset(self, landmarks):
193     try:
194         left_center = self._calculate_eye_center(landmarks, self.
195             LEFT_EYE_INDICES)
196         left_iris = landmarks.landmark[468]
197
198         right_center = self._calculate_eye_center(landmarks, self.
199             RIGHT_EYE_INDICES)
200         right_iris = landmarks.landmark[473]
201
202         offset_x = ((left_iris.x - left_center[0]) + (right_iris.x -
203             right_center[0])) / 2
204         offset_y = ((left_iris.y - left_center[1]) + (right_iris.y -
205             right_center[1])) / 2
206
207         return (offset_x, offset_y)
208     except:
209         return (0, 0)
210
211 def _detect_significant_movement(self, current_gaze):
212     current_time = time.time()
213
214     time_since_last_movement = current_time - self.last_movement_time
215     if time_since_last_movement < self.min_movement_interval:
216         return False, {}
217
218     if self.last_valid_gaze is None:
219         self.last_valid_gaze = current_gaze
220         self.gaze_history.append(current_gaze)
221         return True, {'magnitude': 0, 'direction': 0}
222
223     movement_magnitude = math.sqrt(
224         (current_gaze[0] - self.last_valid_gaze[0])**2 +
225         (current_gaze[1] - self.last_valid_gaze[1])**2
226     )

```

```

224     dx = current_gaze[0] - self.last_valid_gaze[0]
225     dy = current_gaze[1] - self.last_valid_gaze[1]
226     movement_direction = math.atan2(dy, dx)
227
228     if movement_magnitude > self.movement_threshold:
229         self.gaze_history.append(current_gaze)
230         return True, {
231             'magnitude': movement_magnitude,
232             'direction': movement_direction
233         }
234
235     return False, {}
236
237 def _calculate_attention_score(self, eye_state):
238     openness_score = eye_state['avg_openness']
239     symmetry_score = eye_state['symmetry']
240     stability_score = 1.0 - min(1.0, self.consecutive_still_frames / 50.0)
241
242     attention_score = (
243         openness_score * 0.4 +
244         symmetry_score * 0.3 +
245         stability_score * 0.3
246     )
247
248     return min(1.0, max(0.0, attention_score))
249
250 def _draw_advanced_visualization(self, frame, landmarks, movement_detected,
251     current_gaze):
252     h, w = frame.shape[:2]
253
254     color = (0, 255, 0) if movement_detected else (0, 165, 255)
255
256     for idx in self.LEFT_EYE_INDICES + self.RIGHT_EYE_INDICES:
257         point = landmarks.landmark[idx]
258         x = int(point.x * w)
259         y = int(point.y * h)
260         cv2.circle(frame, (x, y), 2, color, -1)
261
262     gaze_x = int(current_gaze[0] * w / self.screen_width)
263     gaze_y = int(current_gaze[1] * h / self.screen_height)
264
265     if movement_detected:
266         cv2.circle(frame, (gaze_x, gaze_y), 10, (0, 0, 255), -1)
267         cv2.circle(frame, (gaze_x, gaze_y), 15, (255, 255, 255), 2)
268     else:
269         cv2.circle(frame, (gaze_x, gaze_y), 6, (0, 165, 255), -1)
270
271     center_x, center_y = w // 2, h // 2

```

```

271         cv2.line(frame, (center_x, center_y), (gaze_x, gaze_y), color, 1)
272
273     def _draw_status(self, frame, status, color):
274         stats = self.get_movement_statistics()
275
276         if self.eye_state_history:
277             eye_status = 'OPEN' if self.eye_state_history[-1]['eyes_open'] else '
                CLOSED'
278         else:
279             eye_status = 'UNKNOWN'
280
281         info_text = [
282             f"Status: {status}",
283             f"Movements: {stats['movement_count']}",
284             f"Still Frames: {self.consecutive_still_frames}",
285             f"Eye State: {eye_status}",
286             f>Data Quality: {stats['data_quality']:.1f}%"
287         ]
288
289         for i, text in enumerate(info_text):
290             y_position = 30 + i * 25
291             cv2.putText(frame, text, (10, y_position),
292                         cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 0), 3)
293             cv2.putText(frame, text, (10, y_position),
294                         cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 1)
295
296     def get_movement_statistics(self):
297         movement_count = len(self.movement_events)
298         total_frames = self.frame_count
299
300         if total_frames == 0:
301             return {
302                 'movement_count': 0,
303                 'movement_rate': 0.0,
304                 'data_quality': 0.0,
305                 'avg_movement_magnitude': 0.0
306             }
307
308         magnitudes = [event['movement_magnitude'] for event in self.
            movement_events]
309         avg_magnitude = np.mean(magnitudes) if magnitudes else 0.0
310
311         valid_eye_states = [state for state in self.eye_state_history if state['
            eyes_open']]
312         data_quality = (len(valid_eye_states) / len(self.eye_state_history)) * 100
            if self.eye_state_history else 0.0
313
314         return {

```

```

315         'movement_count': movement_count,
316         'movement_rate': (movement_count / total_frames) * 100,
317         'data_quality': data_quality,
318         'avg_movement_magnitude': avg_magnitude,
319         'total_frames': total_frames
320     }
321
322
323 class MovementBasedCryptoGenerator:
324     """
325     Crypto generator based on eye movements.
326     """
327
328     def __init__(self, tracker):
329         self.tracker = tracker
330         self.entropy_pool = deque(maxlen=1000)
331
332     def extract_movement_entropy(self):
333         if len(self.tracker.movement_events) < 5:
334             return []
335
336         bits = []
337
338         for movement in self.tracker.movement_events:
339             movement_bits = self._extract_movement_bits(movement)
340             timing_bits = self._extract_timing_bits(movement)
341             physiological_bits = self._extract_physiological_bits(movement)
342
343             bits.extend(movement_bits + timing_bits + physiological_bits)
344
345         return bits
346
347     def _extract_movement_bits(self, movement):
348         bits = []
349
350         x_int = int(movement['x'] * 1000) % 256
351         y_int = int(movement['y'] * 1000) % 256
352
353         x_bits = [int(bit) for bit in format(x_int, '08b')[-4:]]
354         y_bits = [int(bit) for bit in format(y_int, '08b')[-4:]]
355
356         magnitude_int = int(movement['movement_magnitude'] * 100) % 16
357         magnitude_bits = [int(bit) for bit in format(magnitude_int, '04b')]
358
359         direction = movement['movement_direction']
360         direction_quantized = int((direction + math.pi) / (2 * math.pi) * 8) % 8
361         direction_bits = [int(bit) for bit in format(direction_quantized, '03b')]
362

```

```

363         bits.extend(x_bits + y_bits + magnitude_bits + direction_bits)
364     return bits
365
366     def _extract_timing_bits(self, movement):
367         bits = []
368
369         timestamp = movement['timestamp']
370         microseconds = int((timestamp - int(timestamp)) * 1e6)
371
372         time_bits = [int(bit) for bit in format(microseconds % 16, '04b')]
373         bits.extend(time_bits)
374
375         return bits
376
377     def _extract_physiological_bits(self, movement):
378         bits = []
379
380         openness_int = int(movement['eye_openness'] * 100) % 8
381         openness_bits = [int(bit) for bit in format(openness_int, '03b')]
382
383         attention_int = int(movement['attention_score'] * 100) % 8
384         attention_bits = [int(bit) for bit in format(attention_int, '03b')]
385
386         bits.extend(openness_bits + attention_bits)
387         return bits
388
389     def generate_secure_bits(self, num_bits=256):
390         movement_count = len(self.tracker.movement_events)
391
392         if movement_count < 20:
393             print(f"Not enough movements: {movement_count}/20")
394             return []
395
396         print(f"Generating {num_bits} bits from {movement_count} movements...")
397
398         bits = []
399         attempts = 0
400         max_attempts = 50
401         byte_to_gen_key = bytearray()
402         while len(bits) < num_bits and attempts < max_attempts:
403             new_bits = self.extract_movement_entropy()
404             if new_bits:
405                 bits.extend(new_bits)
406
407             attempts += 1
408
409             if len(bits) < num_bits:
410                 time.sleep(0.1)

```

```

411
412     timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
413     filepath = f"newbits_data_{timestamp}.bin"
414     with open(filepath, "wb") as f:
415         for i in range(0, len(bits), 8):
416             chunk = bits[i:i+8]
417             if len(chunk) == 8:
418                 byte_value = int(''.join(map(str, chunk)), 2)
419                 f.write(bytes([byte_value]))
420             else:
421                 incomplete_byte = chunk + [0] * (8 - len(chunk))
422                 byte_value = int(''.join(map(str, incomplete_byte)), 2)
423                 byte_to_gen_key.append(byte_value)
424                 f.write(byte_to_gen_key)
425
426     sha256_hash = hashlib.sha256(byte_to_gen_key).digest()
427     num_bytes = num_bits // 8
428     final_key = sha256_hash[:num_bytes]
429
430     return final_key
431
432
433 def main():
434     if not MEDIAPIPE_AVAILABLE:
435         print("MediaPipe not installed. Run: pip install mediapipe")
436         return
437
438     tracker = AdvancedGazeTracker()
439     crypto_generator = MovementBasedCryptoGenerator(tracker)
440
441     cap = cv2.VideoCapture(0)
442     if not cap.isOpened():
443         print("Cannot open camera")
444         return
445
446     cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
447     cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
448     cap.set(cv2.CAP_PROP_FPS, 30)
449
450     print(" CONTROLS:")
451     print(" Q or ESC - Quit")
452     print(" S - Statistics")
453     print(" B - Generate 128 bits")
454     print(" K - Generate 256-bit key")
455     print(" R - Reset data")
456     print(" =" * 60)
457
458     try:

```

```

459 while True:
460     ret, frame = cap.read()
461     if not ret:
462         break
463
464     processed_frame, gaze_data = tracker.process_frame(frame)
465
466     cv2.imshow('Advanced Eye Tracker - Q to quit', processed_frame)
467
468     key = cv2.waitKey(1) & 0xFF
469     if key == ord('q') or key == 27:
470         break
471     elif key == ord('s'):
472         stats = tracker.get_movement_statistics()
473         print("\n STATISTICS:")
474         print(f" Total frames: {stats['total_frames']}")
475         print(f" Movements: {stats['movement_count']}")
476         print(f" Movement rate: {stats['movement_rate']:.1f}%")
477         print(f" Data quality: {stats['data_quality']:.1f}%")
478         print(f" Avg movement magnitude: {stats['avg_movement_magnitude']:.1
            f}px")
479     elif key == ord('b'):
480         print("\n Generating 128-bit key...")
481         key = crypto_generator.generate_secure_bits(128)
482         if key and len(key) == 16:
483             with open("eye_crypto_key_128.bin", "wb") as f:
484                 f.write(key)
485                 print("Key saved to eye_crypto_key_128.bin")
486         else:
487             print(" Failed to generate key")
488     elif key == ord('k'):
489         print("\n Generating 256-bit cryptographic key...")
490         key = crypto_generator.generate_secure_bits(256)
491         if key and len(key) == 32:
492             with open("eye_crypto_key_256.bin", "wb") as f:
493                 f.write(key)
494
495                 print(f" Key saved: {len(key)} bytes")
496                 print(f" HEX: {key.hex()[:32]}...")
497         else:
498             print(f" Failed to generate 256-bit key")
499     elif key == ord('r'):
500         tracker.movement_events.clear()
501         tracker.gaze_history.clear()
502         tracker.eye_state_history.clear()
503         tracker.consecutive_still_frames = 0
504         tracker.last_movement_time = 0
505         print(" Data reset")

```

```

506
507 except KeyboardInterrupt:
508     print("\n Interrupted by user")
509 except Exception as e:
510     print(f"\n Error: {e}")
511     import traceback
512     traceback.print_exc()
513 finally:
514     cap.release()
515     cv2.destroyAllWindows()
516
517 stats = tracker.get_movement_statistics()
518 print(f"\n FINAL STATISTICS:")
519 print(f" Total frames: {stats['total_frames']}")
520 print(f" Movements: {stats['movement_count']}")
521 print(f" Efficiency: {stats['movement_rate']:.1f}%")
522 print(f" Data quality: {stats['data_quality']:.1f}%")
523
524 if stats['movement_count'] >= 50:
525     print("\n Automatic final key generation...")
526     key = crypto_generator.generate_secure_bits(256)
527     if key and len(key) == 32:
528         with open("final_eye_key_256.bin", "wb") as f:
529             f.write(key)
530         print(f" Final key saved: {len(key)} bytes")
531
532
533 if __name__ == "__main__":
534     main()

```

Листинг модуля сбора голосовой энтропии (VoiceEntropy.py)

Листинг 4.2: Файл VoiceEntropy.py

```
1 import time
2 import math
3 import hashlib
4 import threading
5 from collections import deque
6 from typing import List, Tuple, Optional
7
8 import numpy as np
9
10 try:
11     import sounddevice as sd
12     SOUNDDEVICE_AVAILABLE = True
13 except ImportError:
14     SOUNDDEVICE_AVAILABLE = False
15     print("sounddevice not installed. Run: pip install sounddevice")
16
17 SAMPLE_RATE = 44100
18 FRAME_SIZE = 2048
19 HOP_SIZE = 512
20 N_MFCC = 13
21 N_FFT_BINS = 16
22 VOICE_LO_HZ = 80
23 VOICE_HI_HZ = 3400
24 MIN_FRAMES = 20
25 NOISE_FLOOR = 0.002
26
27
28 def is_microphone_available() -> bool:
29     if not SOUNDDEVICE_AVAILABLE:
30         return False
31     try:
32         devices = sd.query_devices()
33         for dev in devices:
34             if dev["max_input_channels"] > 0:
35                 return True
36     except Exception:
37         return False
38
39
40
41 def _mel_filterbank(sr: int, n_fft: int, n_mels: int = 26,
42                    fmin: float = 80.0, fmax: float = 3400.0) -> np.ndarray:
```

```

43 def hz_to_mel(f): return 2595 * math.log10(1 + f / 700)
44 def mel_to_hz(m): return 700 * (10 ** (m / 2595) - 1)
45
46 mel_lo = hz_to_mel(fmin)
47 mel_hi = hz_to_mel(fmax)
48 mel_points = np.linspace(mel_lo, mel_hi, n_mels + 2)
49 hz_points = np.array([mel_to_hz(m) for m in mel_points])
50 bin_points = np.floor((n_fft + 1) * hz_points / sr).astype(int)
51
52 n_bins = n_fft // 2 + 1
53 fb = np.zeros((n_mels, n_bins))
54 for m in range(1, n_mels + 1):
55     lo, mid, hi = bin_points[m-1], bin_points[m], bin_points[m+1]
56     for k in range(lo, mid):
57         if mid != lo:
58             fb[m-1, k] = (k - lo) / (mid - lo)
59     for k in range(mid, hi):
60         if hi != mid:
61             fb[m-1, k] = (hi - k) / (hi - mid)
62 return fb
63
64
65 def _dct_matrix(n: int) -> np.ndarray:
66     k = np.arange(n)
67     m = np.arange(n)
68     return np.cos(np.pi * m[:, None] * (2 * k[None, :] + 1) / (2 * n))
69
70
71 _FB = _mel_filterbank(SAMPLE_RATE, FRAME_SIZE, n_mels=26)
72 _DCT = _dct_matrix(26)[:N_MFCC]
73
74
75 def compute_mfcc(frame: np.ndarray) -> np.ndarray:
76     windowed = frame * np.hanning(len(frame))
77     spectrum = np.abs(np.fft.rfft(windowed, n=FRAME_SIZE)) ** 2
78     mel_power = _FB @ spectrum
79     log_mel = np.log(mel_power + 1e-10)
80     mfcc = _DCT @ log_mel
81     return mfcc
82
83
84 def compute_spectral_entropy_bits(frame: np.ndarray) -> List[int]:
85     windowed = frame * np.hanning(len(frame))
86     fft_full = np.fft.rfft(windowed, n=FRAME_SIZE)
87
88     freqs = np.fft.rfftfreq(FRAME_SIZE, 1 / SAMPLE_RATE)
89     mask = (freqs >= VOICE_LO_HZ) & (freqs <= VOICE_HI_HZ)
90     voice_fft = fft_full[mask]

```

```

91
92     step = max(1, len(voice_fft) // N_FFT_BINS)
93     bins = voice_fft[::step][:N_FFT_BINS]
94
95     bits: List[int] = []
96
97     for c in bins:
98         amp = int(abs(c) * 1000) & 0xFF
99         bits += [(amp >> i) & 1 for i in range(3)]
100
101         phase_int = int((math.atan2(c.imag, c.real) + math.pi) / (2 * math.pi) *
102             1000) & 0xFF
103         bits += [(phase_int >> i) & 1 for i in range(3)]
104
105     return bits
106
107 def compute_zcr_bits(frame: np.ndarray) -> List[int]:
108     signs = np.sign(frame)
109     zcr = int(np.sum(np.abs(np.diff(signs))) / 2) % 16
110     return [(zcr >> i) & 1 for i in range(4)]
111
112
113 def compute_jitter_bits(frame: np.ndarray) -> List[int]:
114     threshold = max(np.max(np.abs(frame)) * 0.3, NOISE_FLOOR)
115     peaks = []
116     for i in range(1, len(frame) - 1):
117         if frame[i] > threshold and frame[i] > frame[i-1] and frame[i] > frame[i
118             +1]:
119             peaks.append(i)
120
121     if len(peaks) < 3:
122         return [0, 0, 0, 0]
123
124     intervals = np.diff(peaks)
125     jitter = int(np.std(intervals) * 100) % 16
126     return [(jitter >> i) & 1 for i in range(4)]
127
128 def compute_amplitude_bits(frame: np.ndarray) -> List[int]:
129     indices = np.linspace(0, len(frame) - 1, 8, dtype=int)
130     raw = (frame[indices] * 32767).astype(int)
131     return [int(r) & 1 for r in raw]
132
133
134 def compute_mfcc_bits(mfcc: np.ndarray) -> List[int]:
135     bits = []
136     for coeff in mfcc[:N_MFCC-1]:

```

```

137         frac = int(abs(coeff) * 100) % 2
138         bits.append(frac)
139     return bits
140
141
142 class VoiceEntropyCollector:
143     def __init__(self):
144         self._lock = threading.Lock()
145         self._entropy_pool : bytearray = bytearray()
146         self._bit_buffer : List[int] = []
147         self._frame_count : int = 0
148         self._active_frames : int = 0
149         self._running : bool = False
150         self._microphone_confirmed : bool = False
151         self._stream : Optional[object] = None
152         self._sample_buffer = np.zeros(FRAME_SIZE, dtype=np.float32)
153         self._buf_pos : int = 0
154         self._last_stats = {}
155
156         self._amplitude_history : deque = deque(maxlen=60)
157         self._snr_history : deque = deque(maxlen=30)
158         self._entropy_quality : float = 0.0
159
160
161     def start(self) -> bool:
162         if not SOUNDDEVICE_AVAILABLE:
163             return False
164         if self._running:
165             return True
166
167         try:
168             self._stream = sd.InputStream(
169                 samplerate = SAMPLE_RATE,
170                 channels = 1,
171                 dtype = "float32",
172                 blocksize = HOP_SIZE,
173                 callback = self._audio_callback,
174             )
175             self._stream.start()
176             self._running = True
177             self._microphone_confirmed = True
178             print("VoiceEntropyCollector started")
179             return True
180         except Exception as e:
181             self._microphone_confirmed = False
182             print(f"Microphone error: {e}")
183             return False
184

```

```

185
186 def stop(self):
187     self._running = False
188     self._microphone_confirmed = False
189     if self._stream:
190         try:
191             self._stream.stop()
192             self._stream.close()
193         except Exception:
194             pass
195     self._stream = None
196     print("VoiceEntropyCollector stopped")
197
198
199 def reset(self):
200     with self._lock:
201         self._entropy_pool.clear()
202         self._bit_buffer.clear()
203         self._frame_count = 0
204         self._active_frames = 0
205         self._amplitude_history.clear()
206         self._snr_history.clear()
207         self._entropy_quality = 0.0
208
209
210 def _audio_callback(self, indata: np.ndarray, frames: int,
211                     time_info, status):
212     samples = indata[:, 0]
213
214     for s in samples:
215         self._sample_buffer[self._buf_pos] = s
216         self._buf_pos += 1
217
218         if self._buf_pos >= FRAME_SIZE:
219             self._process_frame(self._sample_buffer.copy())
220             self._sample_buffer[:FRAME_SIZE - HOP_SIZE] = \
221                 self._sample_buffer[HOP_SIZE:]
222             self._buf_pos = FRAME_SIZE - HOP_SIZE
223
224
225 def _process_frame(self, frame: np.ndarray):
226     rms = float(np.sqrt(np.mean(frame ** 2)))
227     self._amplitude_history.append(rms)
228
229     self._frame_count += 1
230
231     is_active = rms > NOISE_FLOOR
232     if is_active:

```

```

233         self._active_frames += 1
234
235     bits: List[int] = []
236
237     bits.extend(compute_amplitude_bits(frame))
238     bits.extend(compute_zcr_bits(frame))
239     bits.extend(compute_jitter_bits(frame))
240     bits.extend(compute_spectral_entropy_bits(frame))
241
242     if is_active:
243         mfcc = compute_mfcc(frame)
244         bits.extend(compute_mfcc_bits(mfcc))
245
246     ts_bits = int((time.perf_counter() * 1e6)) & 0xF
247     bits.extend([(ts_bits >> i) & 1 for i in range(4)])
248
249     with self._lock:
250         self._bit_buffer.extend(bits)
251
252         while len(self._bit_buffer) >= 8:
253             byte_bits = self._bit_buffer[:8]
254             del self._bit_buffer[:8]
255             byte_val = sum(b << i for i, b in enumerate(byte_bits))
256             self._entropy_pool.append(byte_val)
257
258     self._update_entropy_quality()
259
260
261     def _update_entropy_quality(self):
262         with self._lock:
263             pool = bytes(self._entropy_pool[-256:])
264
265             if len(pool) < 32:
266                 self._entropy_quality = 0.0
267                 return
268
269             counts = np.bincount(np.frombuffer(pool, dtype=np.uint8), minlength=256)
270             probs = counts / counts.sum()
271             mask = probs > 0
272             H = -np.sum(probs[mask] * np.log2(probs[mask]))
273             self._entropy_quality = H / 8.0
274
275
276     def get_entropy_bits(self) -> List[int]:
277         with self._lock:
278             byte_bits: List[int] = []
279             for byte in self._entropy_pool:
280                 for i in range(8):

```

```

281         byte_bits.append((byte >> i) & 1)
282     return byte_bits
283
284
285     def get_entropy_bytes(self) -> bytes:
286         if not self._microphone_confirmed:
287             return b""
288         with self._lock:
289             return bytes(self._entropy_pool)
290
291
292     def microphone_active(self) -> bool:
293         return self._microphone_confirmed and self._running
294
295
296     def has_enough_entropy(self, min_frames: int = MIN_FRAMES) -> bool:
297         return self._frame_count >= min_frames
298
299
300     def has_enough_voice_activity(self, max_silence_pct: float = 0.40) -> bool:
301         if self._frame_count < MIN_FRAMES:
302             return False
303         silence_pct = (self._frame_count - self._active_frames) / self._frame_count
304         return silence_pct <= max_silence_pct
305
306
307     def silence_percent(self) -> float:
308         if self._frame_count == 0:
309             return 1.0
310         return (self._frame_count - self._active_frames) / self._frame_count
311
312
313     def get_statistics(self) -> dict:
314         with self._lock:
315             pool_size = len(self._entropy_pool)
316
317             rms_vals = list(self._amplitude_history)
318             avg_rms = float(np.mean(rms_vals)) if rms_vals else 0.0
319             peak_rms = float(np.max(rms_vals)) if rms_vals else 0.0
320
321             def to_db(rms): return 20 * math.log10(rms + 1e-10)
322             avg_db = to_db(avg_rms)
323             peak_db = to_db(peak_rms)
324
325             signal_rms = max(rms_vals) if rms_vals else 0.0
326             noise_rms = min(rms_vals) if rms_vals else 1e-10
327             snr_db = to_db(signal_rms / (noise_rms + 1e-10)) if noise_rms > 0 else 0.0

```

```

328
329     dominant_freq = 0.0
330     if rms_vals and avg_rms > NOISE_FLOOR:
331         with self._lock:
332             buf_snap = self._sample_buffer.copy()
333             spectrum = np.abs(np.fft.rfft(buf_snap * np.hanning(FRAME_SIZE))) ** 2
334             freqs = np.fft.rfftfreq(FRAME_SIZE, 1 / SAMPLE_RATE)
335             mask = (freqs >= VOICE_LO_HZ) & (freqs <= VOICE_HI_HZ)
336             if mask.any():
337                 dominant_freq = float(freqs[mask][np.argmax(spectrum[mask])])
338
339     is_active = avg_rms > NOISE_FLOOR
340     quality = min(100.0, self._entropy_quality * 100)
341
342     self._last_stats = {
343         "is_active": is_active,
344         "frame_count": self._frame_count,
345         "active_frames": self._active_frames,
346         "pool_bytes": pool_size,
347         "avg_db": avg_db,
348         "peak_db": peak_db,
349         "snr_db": snr_db,
350         "dominant_freq": dominant_freq,
351         "entropy_quality": quality,
352         "audio_levels": [min(1.0, r / 0.5) for r in rms_vals[-40:]],
353     }
354     return self._last_stats
355
356
357 def combine_entropy(eye_bytes: bytes, voice_bytes: bytes) -> bytes:
358     if not eye_bytes and not voice_bytes:
359         return b""
360
361     eye_len = len(eye_bytes)
362     voice_len = len(voice_bytes)
363
364     pool = bytearray(eye_bytes) + bytearray(voice_bytes)
365
366     return bytes(pool)
367
368
369 def derive_key(raw_pool: bytes, key_bits: int = 256) -> bytes:
370     if len(raw_pool) < 16:
371         return b""
372     sha256_hash = hashlib.sha256(raw_pool).digest()
373     num_bytes = key_bits // 8
374     return sha256_hash[:num_bytes]
375

```

```

376
377 def generate_combined_key(eye_bytes: bytes,
378                           voice_collector: VoiceEntropyCollector,
379                           key_bits: int = 256) -> bytes:
380     voice_bytes = voice_collector.get_entropy_bytes()
381
382     if not eye_bytes and not voice_bytes:
383         print("No data from any source")
384         return b""
385
386     if not eye_bytes:
387         print("No eye data -using only voice")
388         raw_pool = voice_bytes
389     elif not voice_bytes:
390         print("No voice data -using only eyes")
391         raw_pool = eye_bytes
392     else:
393         raw_pool = combine_entropy(eye_bytes, voice_bytes)
394
395     key = derive_key(raw_pool, key_bits)
396     print(f"Key {key_bits} bits generated (eyes: {len(eye_bytes)}B, voice: {len(
397         voice_bytes)}B, pool: {len(raw_pool)}B)")
398     return key

```